

FAILLE Kéryann
SAHAN Burak
LHERITIER Dylan
MALHERBE Garance

Rapport de Projet : Suivi Autonome de Phase de Compost (SAPC)



Professeur Référent : **Monsieur MAYLAENDER**

Lycée La Fayette

11 Quai de Seine, 77430 Champagne-sur-Seine



Lycée La Fayette
Champagne-sur-Seine • Fontaineroux

BTS Cybersécurité Informatique et Réseau Électronique (CIEL)

Option Informatique et Réseau (IR)

Session : 2026

TABLE DES MATIÈRE

1. Introduction.....	6
2. Partie Générale.....	7
2.1. Présentation du projet.....	7
2.2. Contexte et problématique.....	7
2.3. Objectifs du projet.....	8
2.4. Fonctionnalités attendues.....	9
3. Expression du besoin.....	10
4. Analyse du projet.....	11
5. Diagrammes.....	12
6. Contraintes (matériel et logiciel).....	14
8.1 : Étudiant 1 : IHM et API REST (Dylan LHERITIER).....	18
1. Présentation Générale.....	18
1.1 Contexte de réalisation.....	18
1.2 Problématique technique.....	18
1.3 Présentation de la solution retenue.....	18
1.4 Situation du projet dans son environnement (Diagramme de contexte).....	19
2. Analyse du Besoin et Spécifications.....	19
2.1 Analyse fonctionnelle (Diagrammes SysML).....	19
2.1.1 Diagramme des Cas d'Utilisation (Use Case).....	19
2.1.2 Diagramme de Définition de Blocs (BDD).....	20
2.2 Cahier des charges spécifique à ma mission.....	20
2.3 Spécifications techniques.....	20
2.4 Contraintes de développement.....	21
3. Architecture Logicielle et Conception (3 pages).....	21
3.1 Architecture Multi-thread.....	21
3.2 Modélisation et protection des données.....	22
3.2.1 La structure SapcData.....	22
3.2.2 Le mécanisme d'exclusion mutuelle (Mutex).....	22
3.3 Conception de la classe sapc_rest_server.....	22
3.4 Design de l'API REST.....	23
3.4.1 Diagramme de séquence :.....	24
3.5 Format d'échange : JSON.....	25
4. Réalisation Technique - Le Backend (4 pages).....	25
4.1 Développement du Serveur REST.....	25
4.1.1 Initialisation et Constructeur.....	25
4.1.2 Configuration des routes (Routing).....	26
4.2 Gestion des Handlers et formatage JSON.....	26
5. RÉALISATION TECHNIQUE : L'INTERFACE WEB (FRONTEND).....	28

5.1 Architecture et Technologies Web.....	28
5.2.1 Récupération des données (Requête GET).....	30
5.3 Organisation du Dashboard.....	30
5.4 Gestion des alertes visuelles.....	30
6. Tests, Validation et Maintenance (2 pages).....	30
6.1 Protocole de tests unitaires (En cours).....	30
6.2 Tests d'intégration (Prévus).....	31
6.3 Difficultés rencontrées et solutions.....	31
7. Conclusion.....	31
7.1 Bilan technique.....	31
7.2 Bilan personnel.....	32
7.3 Perspectives d'évolution.....	32
8.2 : Étudiant 2 : MCP3008 (Système de mesure d'humidité et de température)(Keryann FAILLE).....	33
8.2.1. Introduction et Problématique.....	33
8.2.2. Étude des Solutions Matérielles.....	34
8.2.3. Bus de Communication SPI.....	36
8.2.4. Architecture Électrique et Adaptation.....	37
C. Le calcul du pas de mesure (Quantum).....	38
8.2.5. Développement Logiciel en C++.....	39
8.2.7. Conclusion et Perspectives.....	43
8.3 : Étudiant 3 : INA226 (Système de mesure de tension, courant et puissance)(Burak Sahan).....	44
Développement technique : mesure de tension et de courant.....	44
8.3.1. Analyse globale de ma mission de métrologie énergétique.....	44
8.3.2. Justification technique du choix du composant INA226.....	45
8.3.3 Étude approfondie de la mesure par résistance shunt.....	46
Le phénomène physique de la chute de tension.....	46
Le choix stratégique de la résistance de 0,1 Ohm.....	46
Calculs théoriques et validité du montage.....	46
8.3.4 Architecture de la chaîne d'alimentation photovoltaïque.....	47
Le captage : Panneau solaire monocristallin de 80W.....	47
La régulation : Le contrôleur MPPT.....	47
Le stockage : Batterie LiFePO4 (Lithium Fer Phosphate).....	48
La conversion et distribution : Convertisseur DC/DC TRACO Power.....	48
L'intégration de la métrologie (INA226).....	48
8.3.5 Conception de la classe métier INA226 en langage C++.....	49
Structure de la classe et encapsulation.....	49
Gestion de la communication I2C sous Linux.....	50
Logique de calibration et de conversion.....	50
Méthodes de lecture et gestion des données physiques.....	50
Gestion des erreurs et robustesse.....	51

8.3.6	Organisation et planification du projet (Diagramme de Gantt).....	51
8.3.7	Présentation du diagramme UML.....	52
	Analyse des composants de la classe.....	52
	Justification de ce choix d'architecture.....	53
8.3.8	Conception logicielle : Diagramme de classe de l'unité INA226.....	53
	Implémentation logicielle et développement des méthodes de calcul.....	54
	Gestion de l'interface matérielle (Bus I2C).....	54
	Lecture des registres et traitement des données 16 bits.....	54
	Algorithmes de conversion des grandeurs physiques.....	55
	Intégration temporelle et calcul de l'énergie (Wh).....	55
	Estimation de l'autonomie réelle.....	56
8.3.9	Algorithmes de calcul de puissance et d'énergie cumulée.....	56
	La puissance instantanée : le produit $U * I$	56
	Détail de l'implémentation : la méthode des rectangles.....	57
	Justification et précision de l'algorithme.....	57
8.3.10	Méthodologie d'estimation de l'autonomie restante.....	57
1.	Le principe de calcul : calcul de l'autonomie réelle.....	58
2.	Intégration du coefficient de sécurité et d'efficacité.....	58
3.	Traduction algorithmique en C++.....	59
4.	Analyse de la précision et limites.....	59
8.3.11	Stratégie de mise en œuvre et résolution des problèmes techniques.....	60
8.3.12	Validation expérimentale et métrologie comparative.....	60
8.3.13	Bilan personnel et perspectives d'évolution du système.....	61
7.4.	Étudiant 4 Base de données (BDD) (MALHERBE Garance).....	61
7.4.1.	Introduction.....	61
7.4.2.	Diagramme MCD.....	62
7.4.3.	Développement.....	63
7.4.4.	Conclusion – Étudiant 4.....	64
8.	Conclusion Générale.....	65
9.	GLOSSAIRE.....	65

1. Introduction

Ce dossier s'inscrit dans le cadre de notre projet de fin de Deuxième année de **BTS CIEL (Cybersécurité Informatique et Réseau Électronique) Option IR (Informatique et Réseau)**. Ce projet a pour objectif de mettre en pratique les compétences acquises durant la formation à travers la réalisation d'un système complet, de la conception jusqu'à la mise en œuvre.



Le projet étudié nommé **Suivi Autonome de Phase de Compost (SAPC)**, consiste à développer un dispositif permettant de surveiller à distance l'évolution du compost. Ce système repose sur l'utilisation de capteurs pour mesurer la température, l'humidité ainsi que la consommation énergétique, le tout piloté par un **Raspberry Pi 3** et alimenté de manière autonome grâce à l'énergie solaire.

Les données collectées sont traitées stockées dans une **Base De Données (BDD)**, puis accessibles via une interface web. Un système d'alerte est également mis en place afin de prévenir les utilisateurs en cas de conditions particulières nécessitant une intervention.

Dans ce dossier nous allons présenter :

- ☐ **Le contexte du projet,**
- ☐ **L'expression du besoin,**
- ☐ **L'analyse technique,**
- ☐ **Les contraintes à respecter,**
- ☐ **Et la répartition des tâches au sein de l'équipe.**

2. Partie Générale

2.1. Présentation du projet

Le projet consiste à concevoir un système nommé **Suivi Autonome de Phase de Compost (SAPC)**. Il s'inscrit dans une démarche écologique visant à améliorer la gestion des déchets organiques au sein de notre établissement scolaire. Dans ce cadre, des bacs à compost ont été installés sur les deux sites de l'établissement, afin de récupérer les déchets alimentaires des élèves et de produire du compost réutilisable pour l'entretien des espaces verts des communes voisines.

Ce projet a pour but d'apporter une solution technologique et écologique permettant d'accompagner et d'optimiser ce processus naturel grâce à un système de supervision autonome.

2.2. Contexte et problématique

Le compostage est un processus biologique naturel qui se déroule en quatre grandes phases :

1. Phase **Mésophile** (Le démarrage) :
 - Acteurs : Micro-organismes aimant la tiédeur (20-45 °C).
 - Action : Décomposition des molécules simples.
 - Signe particulier : La température monte en 2 à 8 jours et le milieu devient acide.
2. Phase **Thermophile** (L'assainissement) :
 - Acteurs : Micro-organismes résistants à la chaleur (45-70 °C).
 - Action : Dégradation des matières complexes (bois, paille).
 - Signe particulier : La chaleur tue les bactéries nocives (**pasteurisation**). Le milieu devient **alcalin** et nécessite beaucoup d'oxygène (aération).
3. Phase de **Refroidissement** (La transition) :
 - Acteurs : Retour des micro-organismes **mésophiles**.
 - Action : Finition du travail sur la cellulose et la **lignine** restantes.
 - Signe particulier : La température chute car les « carburants » (Carbone/Azote) s'épuisent. Le **pH** se stabilise.

4. Phase de **Maturation** (La stabilisation) :

- Acteurs : Faune du sol et réaction chimiques lentes.
- Action : Création de l'**humus** par condensation et **polymérisation**.
- Signe particulier : Se fait à température ambiante. C'est l'étape finale qui transforme les déchets en véritable compost stable.

Chacune de ces phases se caractérise par des variations de température et d'humidité, qui sont des indicateurs essentiels du bon déroulement du processus.

Cependant, ces variations peuvent également entraîner des dégagements de gaz potentiellement corrosifs ou dangereux. De plus, sans suivi précis, il est difficile de savoir à quel moment intervenir (mélange, ajout de matière, etc.).

La problématique est donc de pouvoir surveiller efficacement et en continu l'état du compost, tout en limitant les interventions humaines directes.

2.3. Objectifs du projet

Le projet SAPC vise à concevoir un système de supervision 100 % autonome pour un composteur urbain connecté, articulé autour de deux axes majeurs :

1. Le suivi biologique du compost (Objectif environnemental)

Mission : Mesurer en continu la température et l'humidité via le module d'acquisition Mcp3008Spi.

Enjeu : Assurer le maintien des conditions idéales de fermentation (phase thermophile entre 55 °C et 65 °C, humidité entre 40 % et 65 %) pour éviter l'arrêt du processus ou l'apparition de mauvaises odeurs.

Action : Alerter l'exploitant via l'IHM en cas de dérive des seuils.

2. L'autosuffisance énergétique (Contrainte et gestion)

Infrastructure matérielle : Le système est isolé du réseau électrique. Il s'alimente via un panneau solaire de 80 W, un régulateur MPPT et une batterie LiFePO4 (choisie pour sa longévité et sa stabilité thermique).

Autosurveillance (Métrologie) : Le module Ina226 calcule en temps réel la puissance et l'énergie (Wh) consommées par le Raspberry Pi (le contrôleur central).

Intelligence embarquée : En cas de faible autonomie, un algorithme prédictif bascule le système en mode de consommation dégradé (lecture des capteurs espacée, coupure des radios gourmandes).

Optimisation logicielle : L'utilisation d'une architecture API REST et de trames JSON légères réduit le temps d'émission réseau, minimisant ainsi l'impact énergétique global.

2.4. Fonctionnalités attendues

L'architecture logicielle repose sur trois piliers fonctionnels interdépendants :

1. Acquisition et stockage des données (Le socle)

Collecte & Filtrage : Le Raspberry Pi (contrôleur central) interroge en continu les capteurs environnementaux (Mcp3008Spi) et météorologiques (Ina226). Les bruits de mesure sont éliminés par traitement algorithmique.

Persistance : Les données validées sont enregistrées par la classe `sapc_bdd` dans une base SQLite.

Objectif : Constituer un historique fiable sur plusieurs semaines pour analyser le cycle du compost et le bilan énergétique.

2. Supervision à distance ubiquitaire (L'IHM)

Architecture Client-Serveur : Le composant `sapc_rest_server` expose une API REST réseau.

Échanges asynchrones : L'IHM web exécute des requêtes JavaScript. Le serveur extrait les données de manière sécurisée (via un mutex) et répond au format JSON.

Design adaptatif (Responsive) : Les données s'affichent sous forme de graphiques et jauges dynamiques, optimisés sans distinction pour ordinateurs ou smartphones.

3. Veille et alertes automatisées (La réactivité)

Analyse en continu : Un algorithme compare en temps réel les mesures de la structure SapcData avec des seuils prédéfinis.

Types d'événements surveillés :

Anomalies critiques : Chute de tension de la batterie, baisse anormale de température (asphyxie du compost).

Transitions biologiques : Passage à la phase de maturation (nécessitant un retournement de la matière).

Notification : Envoi automatique d'un courriel documenté (date, heure, défaut) via le protocole SMTP pour déclencher l'intervention de l'exploitant.

3. Expression du besoin

Le projet SAPC répond au besoin fondamental de s'affranchir d'une surveillance humaine quotidienne et contraignante sur le site du composteur en automatisant la collecte des données. En remplaçant les relevés manuels fastidieux et sujets aux erreurs par un flux métrologique continu, le système permet de passer d'une maintenance subie à une gestion prédictive à distance. Pour cela, le dispositif numérise et centralise la température et l'humidité du compost via le circuit Mcp3008Spi afin de valider biologiquement la phase d'hygiénisation de la matière organique. En parallèle, il assure une autosurveillance de sa propre consommation électrique grâce au module Ina226, collectant les tensions et courants pour estimer son autonomie. L'ensemble de ces informations environnementales et énergétiques alimente ensuite une interface utilisateur adaptative pour la visualisation des courbes, tout en activant un système d'alerte automatisé par courriel en cas de franchissement de seuil ou de transition biologique.

L'implémentation de ces fonctions doit néanmoins composer avec des contraintes matérielles strictes, notamment l'utilisation imposée d'un Raspberry Pi 3 configuré comme le microcontrôleur central. Ce choix technique nécessite le développement de classes en langage C++ pour piloter directement les bus de communication I2C et SPI nécessaires aux capteurs. De plus, pour des raisons de sécurité et d'infrastructure, l'accès aux données doit s'effectuer exclusivement à travers le réseau local de l'établissement via un serveur REST autonome, excluant d'office les services cloud. Enfin, le système garantit une fiabilité et une traçabilité totales des informations en protégeant les accès par mot de passe et en appliquant un horodatage systématique lors de l'enregistrement des données dans la base SQLite, dont les transactions sécurisées préviennent toute perte de mémoire en cas de panne de courant.

4. Analyse du projet

Le projet repose sur une architecture de type système embarqué communicant. Il s'articule autour de plusieurs fonctions techniques qui permettent d'assurer le bon fonctionnement global.

Dans un premier temps, les capteurs collectent les données physiques telles que la température, l'humidité et le courant. Ces données sont ensuite transmises au **Raspberry**

Pi3 qui assure leur traitement. Le système effectue ensuite des calculs notamment pour estimer l'autonomie énergétique.



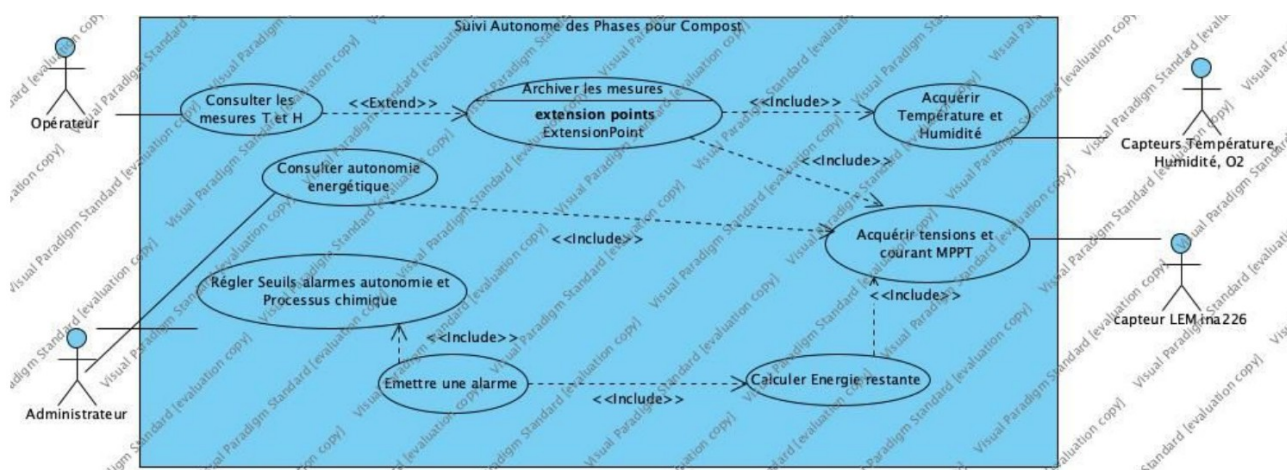
Les données traitées sont stockées dans une base de données local afin de conserver un historique exploitable. Elles sont ensuite mises à disposition via un serveur web permettant leur consultation depuis un ordinateur ou un appareil mobile connecté au réseau local.

Enfin, une interface utilisateur permet d'afficher les informations de manière claire et d'interagir avec le système. Cette organisation permet de structurer le projet autour d'une chaîne d'information complète (acquisition, traitement, stockage et restitution).

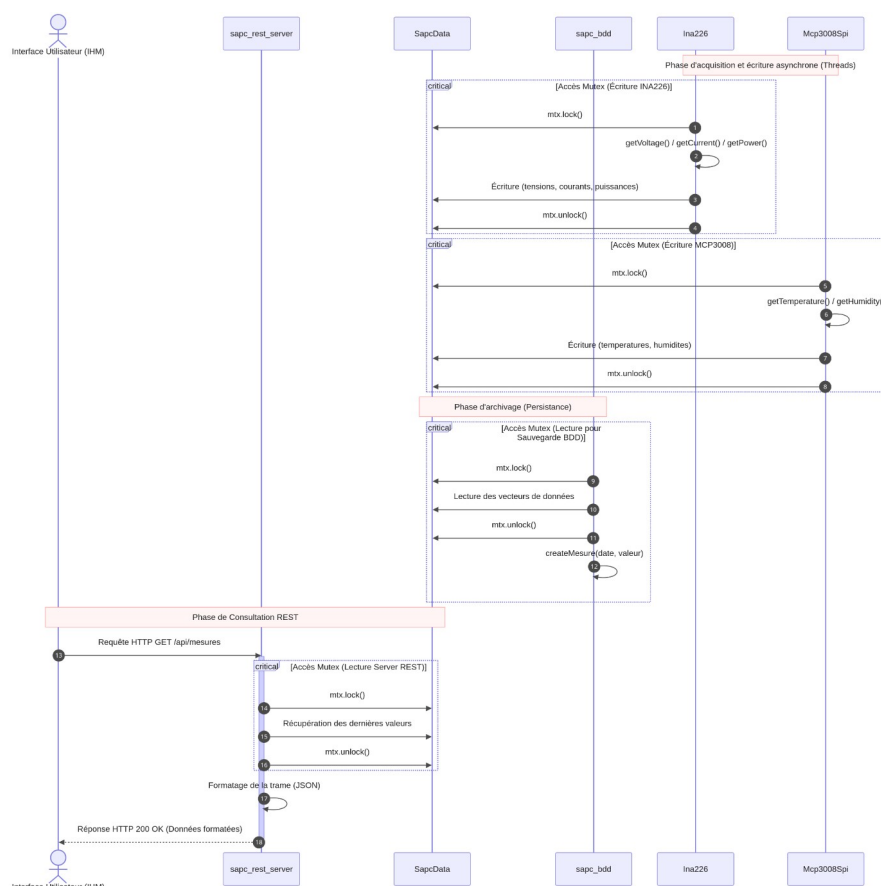
5. Diagrammes

Le projet peut être représenté à l'aide de plusieurs diagrammes issus du **formalisme SYSML**.

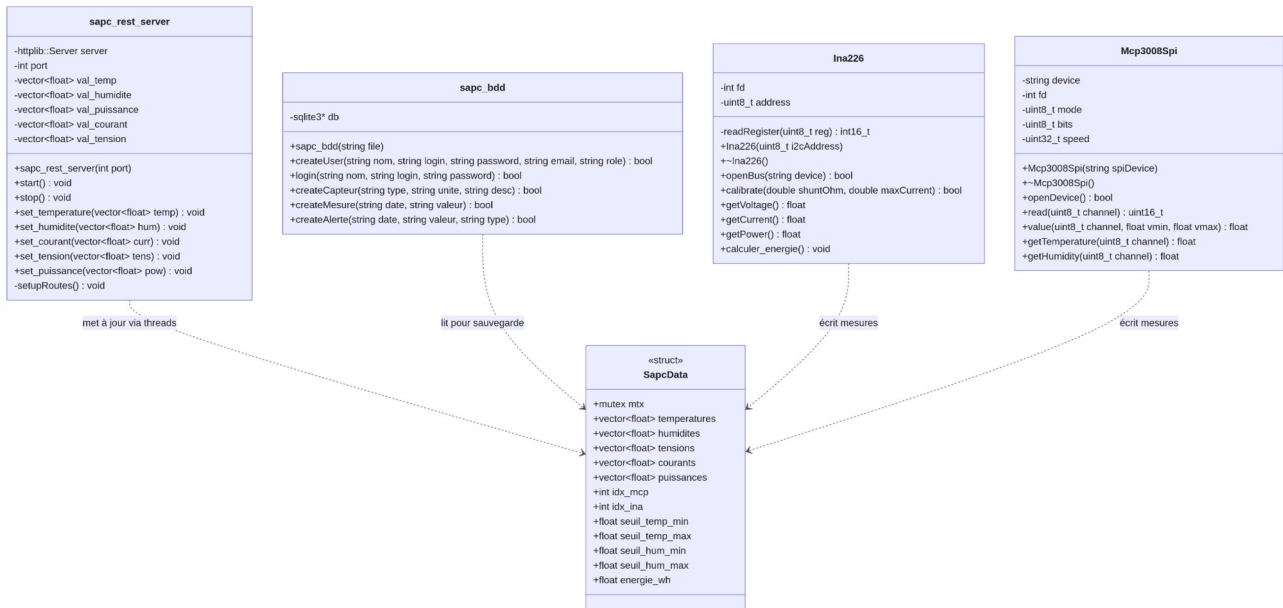
Le diagramme de cas d'utilisation met en évidence les interactions entre les utilisateurs et le système. L'utilisateur peut consulter les mesures, recevoir des alertes et configurer les paramètres du système.



Le diagramme de déploiement décrit l'organisation matérielle. Le **Raspberry Pi3** est placé dans un boîtier étanche à proximité du compost, accompagné des capteurs. Le système est relié au réseau local, permettant l'accès aux données depuis différents terminaux internes.

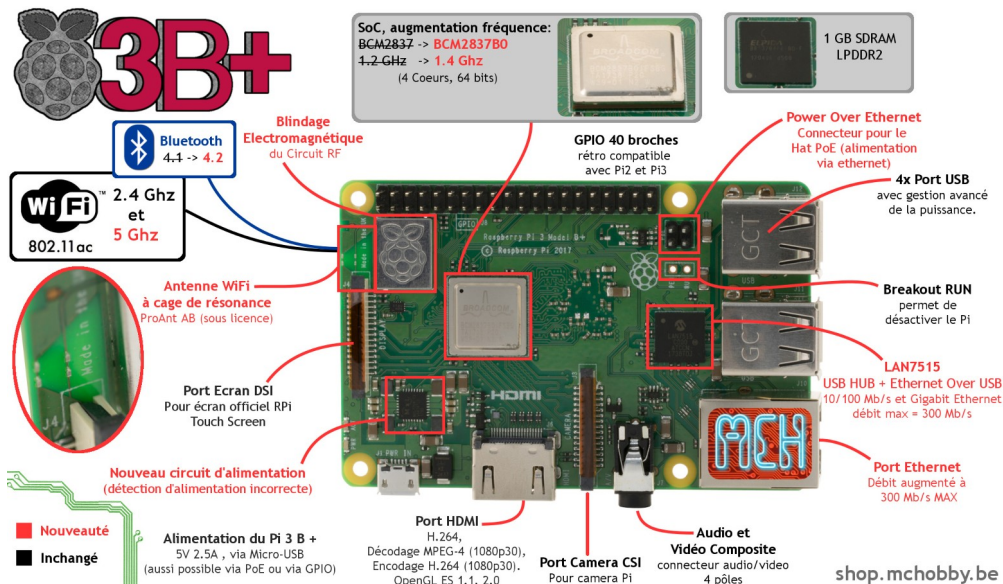


Le **diagramme de classe** permet de modéliser les différentes parties logicielles. Il comprend notamment des classes pour la gestion des capteurs, de la **BDD**, du serveur web et de l'interface utilisateur. Ces classes communiquent entre elles pour assurer le fonctionnement global du système.



6. Contraintes (matériel et logiciel)

Le projet doit respecter plusieurs contraintes techniques importantes. Sur le plan matériel, il est imposé d'utiliser un **Raspberry PI3** ainsi que des capteurs spécifiques pour mesurer la température, l'humidité, le courant ainsi que l'intensité. Un convertisseur analogique-numérique est également nécessaire pour interfacer certains capteurs.



Le système doit être alimenté par un panneau solaire associé à une batterie et à un régulateur de charge de type **MPPT**. Cela garantit une autonomie énergétique, même en cas de faible ensoleillement sur une courte durée.

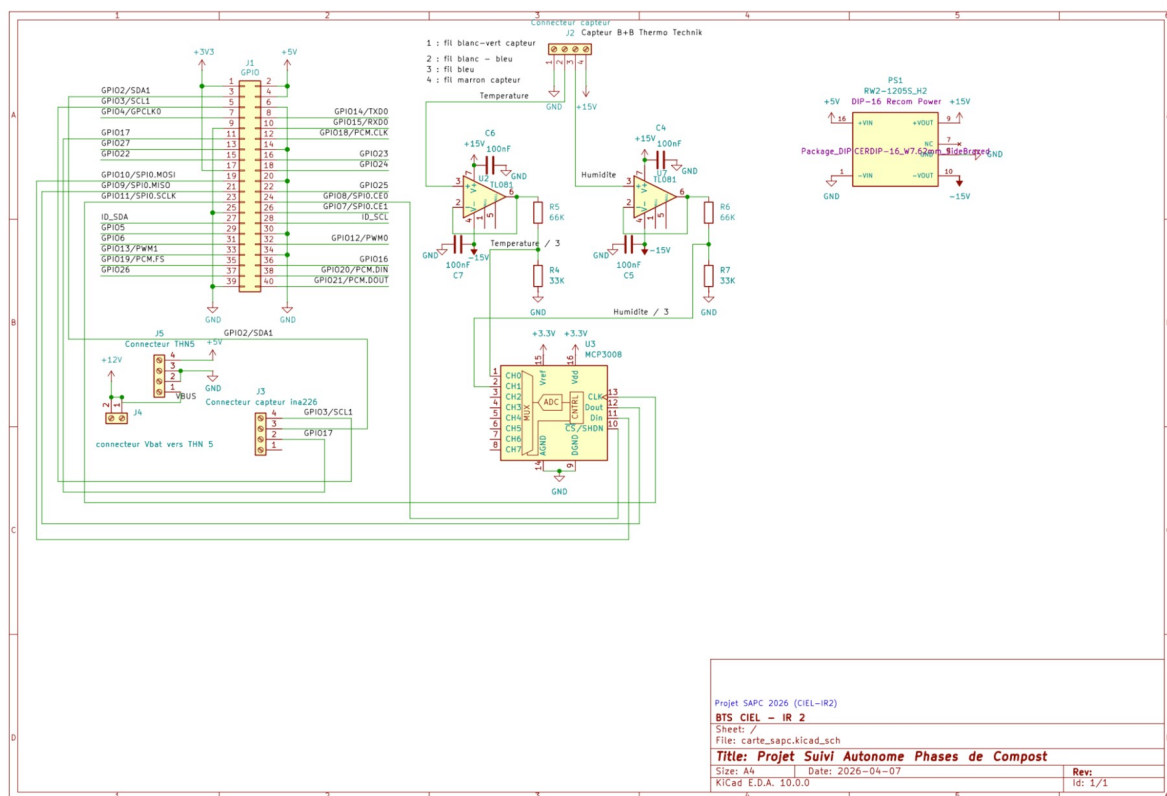


Contrôleur de charge solaire
MPPT 75/15

Sur le plan logiciel, le développement doit être réalisé en langage C++. La gestion des données repose sur une base **SQLite**, tandis que la communication avec les utilisateurs s'effectue via un serveur WEB utilisant une architecture **REST** et des échanges au format **JSON**.

D'autres contraintes doivent également être respectées, notamment un budget limité, des délais de réalisation imposés, ainsi que des exigences en matière de sécurité et de fiabilité du système.

6.1 Schéma de câblage



7. Répartition des tâches

Le projet est réalisé avec une équipe de 4 Étudiants, chacun étant responsable d'une partie spécifique du système :

Le premier Étudiant est chargé de développer l'**Interface Humain Machine (IHM)** ainsi que le serveur web. Il doit également mettre en place les mécanismes d'alerte et assurer la sécurité des communications sur le réseau.

- Le deuxième étudiant s'occupe de l'acquisition des données de température et d'humidité. Il doit étudier les capteurs, réaliser les branchements nécessaires et développer les programmes permettant de récupérer les mesures.
- Le troisième Étudiant est responsable de la mesure du courant et du calcul de l'autonomie énergétique. Il doit exploiter les capteurs dédiés et mettre en place les traitements nécessaires pour analyser la consommation du système.

Le quatrième étudiant prend en charge la **BDD (Base De Données)**. Il doit concevoir le modèle de données, implémenter la base **SQLite** et développer les fonctions permettant de lire et d'écrire les informations.

L'ensemble de l'équipe participe à l'intégration finale du système, aux tests et à la rédaction de la documentation technique.

8.1 : Étudiant 1 : IHM et API REST (Dylan LHERITIER)

1. Présentation Générale

1.1 Contexte de réalisation

Le projet SAPC (Suivi Autonome de Phases de Compost) s'inscrit dans une démarche de développement durable et d'économie circulaire. Il a été initié au sein du Lycée La Fayette en collaboration avec le SMICTOM de Fontainebleau.

L'enjeu majeur est la valorisation des biodéchets (restes de cantine, déchets verts) par le compostage. Ce processus naturel de décomposition de la matière organique par des micro-organismes permet de produire un amendement riche pour les sols. Cependant, pour garantir un compost de qualité et éviter les nuisances (odeurs, prolifération de bactéries pathogènes), une surveillance rigoureuse des conditions physico-chimiques est indispensable.

1.2 Problématique technique

Le compostage se déroule en plusieurs phases critiques :

- La phase hydrolytique : Décomposition initiale.
- La phase thermophile : La température doit monter entre 55°C et 70°C pour hygiéniser le compost (destruction des graines de mauvaises herbes et des germes).
- La phase mésophile : Refroidissement et maturation.

Le problème : Les sites de compostage sont souvent situés dans des zones isolées du lycée, sans accès facile au réseau électrique ou informatique filaire. De plus, l'environnement est agressif (humidité proche de 100%, dégagement de gaz corrosifs comme l'ammoniac).

Le projet SAPC doit donc répondre à trois contraintes :

1. **L'autonomie énergétique** : Fonctionnement sur panneau solaire et batterie.
2. **La mobilité** : Transmission des données sans fil via le réseau local.
3. **La robustesse** : Capteurs et électronique protégés.

1.3 Présentation de la solution retenue

La solution SAPC consiste en une borne de mesure autonome architecturée autour d'un Raspberry Pi. Le système assure la collecte, le stockage et la diffusion des données de manière transparente pour l'utilisateur final.

Le système se décompose en plusieurs sous-systèmes :

- Acquisition analogique : Utilisation d'un convertisseur MCP3008 (SPI) pour les sondes de température et d'humidité.

- Gestion énergétique : Utilisation d'un capteur INA226 (I2C) pour surveiller la charge de la batterie et la consommation.
- Stockage local : Base de données SQLite pour l'historique.
- Interface de consultation : Serveur REST en C++ et Site Web dynamique.

1.4 Situation du projet dans son environnement (Diagramme de contexte)

Le système interagit avec plusieurs acteurs et éléments extérieurs. Le personnel technique du lycée (ou les élèves éco-délégués) consulte les données via un navigateur web. Les capteurs physiques sont en contact direct avec la matière organique, tandis que le panneau solaire fournit l'énergie nécessaire au fonctionnement permanent.

2. Analyse du Besoin et Spécifications

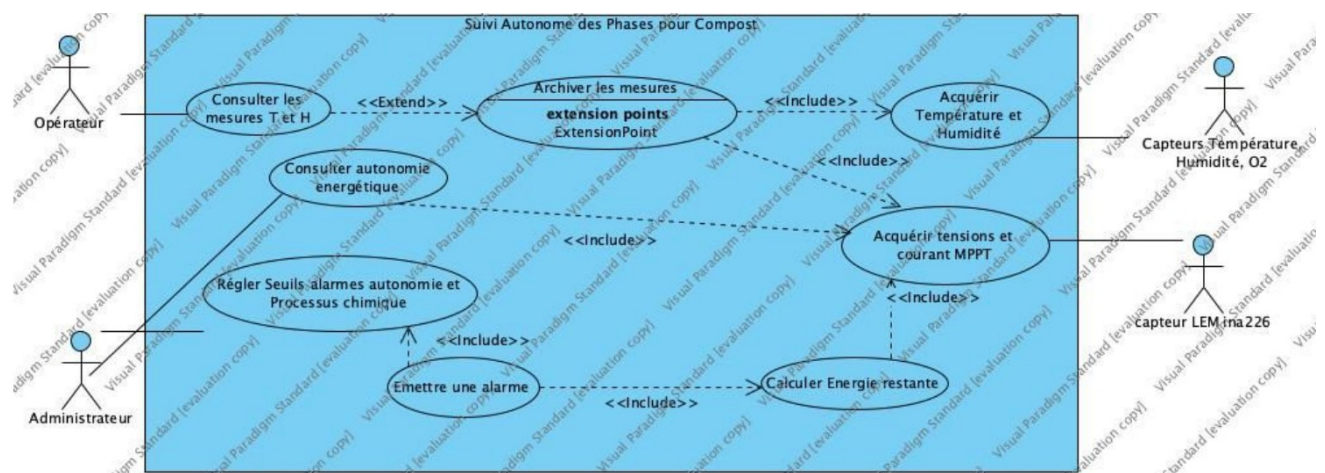
2.1 Analyse fonctionnelle (Diagrammes SysML)

L'analyse du besoin s'appuie sur la méthode SysML pour modéliser les fonctionnalités attendues du système SAPC.

2.1.1 Diagramme des Cas d'Utilisation (Use Case)

Le système doit permettre à l'utilisateur (Personnel technique ou éco-délégué) de :

- **Consulter les mesures** : Visualiser en temps réel la température, l'humidité et l'état énergétique.
- **Paramétrer le système** : Configurer les seuils d'alerte (température critique).
- **Superviser l'autonomie** : Vérifier que la batterie est suffisamment chargée pour assurer la continuité des mesures.



2.1.2 Diagramme de Définition de Blocs (BDD)

Ce diagramme permet de visualiser la hiérarchie du système. La borne SAPC est composée de :

- **Unité de traitement** : Raspberry Pi 4.
- **Bloc Acquisition** : Capteurs MCP3008 (Analogique) et INA226 (I2C).
- **Bloc Communication** : Serveur REST (C++) fournissant des données JSON.
- **Bloc Énergie** : Panneau solaire, régulateur et batterie.

2.2 Cahier des charges spécifique à ma mission

Dans le cadre de ce projet de groupe, mes responsabilités techniques ont été définies selon les exigences suivantes :

1. **Développement d'une API REST** : Créer un service capable de répondre à des requêtes HTTP (GET/POST) pour transmettre les données des capteurs de manière structurée.
2. **Formatage des données** : Utiliser exclusivement le format **JSON** pour garantir la compatibilité avec l'interface web.
3. **Gestion de l'historique** : Le serveur doit maintenir en mémoire tampon les 10 dernières mesures pour permettre l'affichage de graphiques "temps réel".
4. **Interface Web** : Développer une page de supervision ergonomique (Dashboard) accessible depuis n'importe quel navigateur sur le réseau du lycée.

2.3 Spécifications techniques

Caractéristique	Spécification attendue
Langage de programmation	C++ / HTML / CSS / JS
Bibliothèque réseau	cpp-http lib (choisie pour sa légèreté)
Fréquence de rafraîchissement	Toutes les 5 secondes pour le MCP3008 / 10 secondes pour l'INA226

Format d'échange	JSON (JavaScript Object Notation)
Sécurité	Route /login pour restreindre l'accès aux paramètres

2.4 Contraintes de développement

- **Concurrence (Multithreading)** : Le serveur REST ne doit pas bloquer la lecture des capteurs. Il est impératif d'utiliser la bibliothèque `<thread>` de la STL C++.
- **Intégrité des données** : L'accès aux variables partagées (mesures) doit être protégé par des mécanismes d'exclusion mutuelle (**Mutex**) pour éviter les corruptions de mémoire lors de lectures/écritures simultanées.
- **Empreinte mémoire** : Le programme doit être optimisé pour tourner en tâche de fond sur le Raspberry Pi sans saturer les ressources système.

3. Architecture Logicielle et Conception (3 pages)

3.1 Architecture Multi-thread

La borne SAPC doit assurer plusieurs missions simultanément : lire les capteurs à des fréquences différentes, sauvegarder en base de données et répondre aux requêtes des utilisateurs sur le réseau. Pour garantir la réactivité du système, j'ai opté pour une **architecture multi-threadée**.

Le programme principal (main.cpp) orchestre ainsi quatre fils d'exécution (threads) parallèles :

1. **Thread Serveur REST** : Gère les connexions HTTP entrantes.
2. **Thread MCP3008** : Lecture SPI de la température et de l'humidité toutes les 5 secondes.
3. **Thread INA226** : Lecture I2C des paramètres électriques toutes les 10 secondes.
4. **Thread BDD** : Archivage des données et détection des alarmes toutes les 30 secondes.

3.2 Modélisation et protection des données

La communication entre ces threads nécessite un espace de stockage partagé, mais sécurisé.

3.2.1 La structure SapcData

J'ai conçu une structure SapcData qui centralise l'état du système. Elle contient des vecteurs (`std::vector`) de taille fixe (10 éléments) faisant office de "buffers circulaires" pour conserver un historique court des mesures.

3.2.2 Le mécanisme d'exclusion mutuelle (Mutex)

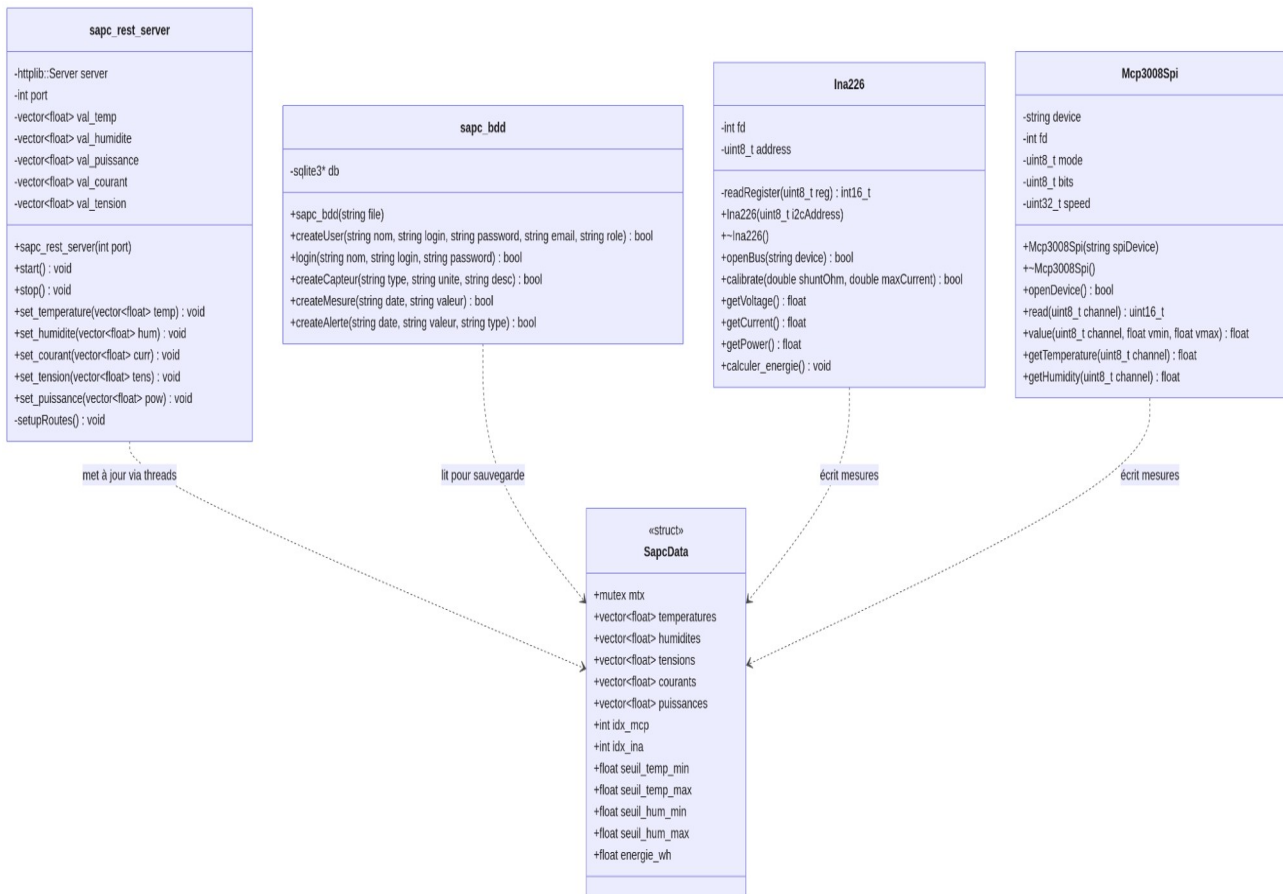
Comme le thread du serveur peut lire une valeur au moment précis où un thread capteur la modifie, il existe un risque de "Race Condition" (concurrence critique). Pour prévenir toute corruption de mémoire, j'ai intégré un **Mutex** (`std::mutex`).

Chaque accès en lecture ou écriture est protégé par un verrou local (`std::lock_guard`), garantissant que l'opération est atomique.

3.3 Conception de la classe `sapc_rest_server`

Pour respecter les principes de la programmation orientée objet (POO), j'ai encapsulé le serveur web dans une classe dédiée.

- **Attributs** : Un objet `httplib::Server` et des conteneurs privés pour les mesures.
- **Méthodes Publiques (Setters)** : Permettent aux threads capteurs de mettre à jour les données du serveur sans exposer directement les variables privées.
- **Méthodes privées (Handlers)** : Fonctions internes déclenchées par les routes HTTP.



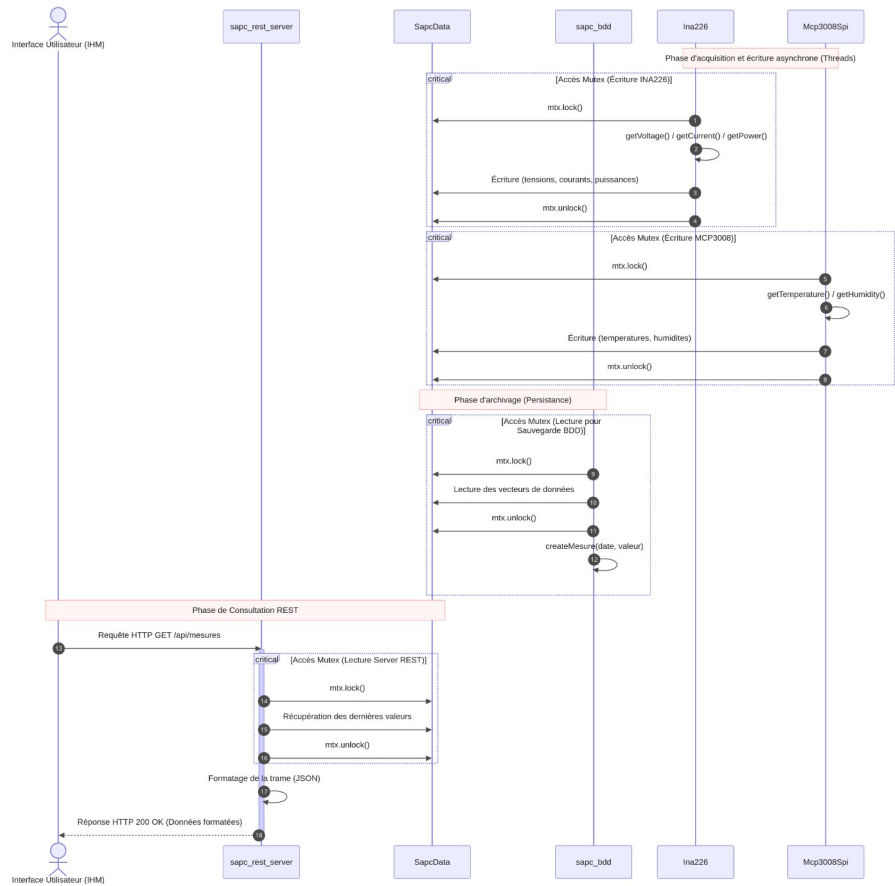
3.4 Design de l'API REST

Le serveur suit les principes du protocole HTTP. J'ai défini un plan d'adressage (endpoints) logique pour faciliter le travail du développeur Frontend (moi-même) :

Endpoint	Méthode	Format de réponse	Description
/mesures/temperature	GET	JSON Array	Liste des 10 dernières températures en °C.
/mesures/humidite	GET	JSON Array	Liste des 10 derniers taux d'humidité en %.
/mesures/energie	GET	JSON Object	Tension (V), Courant (A) et Puissance (W).

/seuils	POST	JSON	Envoi des nouveaux paramètres d'alerte.
---------	------	------	---

3.4.1 Diagramme de séquence :



3.5 Format d'échange : JSON

Le choix du format JSON (JavaScript Object Notation) s'est imposé pour sa légèreté et sa compatibilité native avec les navigateurs web. Chaque mesure est structurée de la manière suivante pour être facilement exploitée par le site web :

JSON

```
{  
  "valeur": 65.4,  
  "unite": "°C",  
  "horodatage": "2026-05-18 14:30:05"  
}
```

4. Réalisation Technique - Le Backend

4.1 Développement du Serveur REST

Le serveur a été développé en C++ en utilisant la bibliothèque `cpp-httpplib`. Mon travail a consisté à transformer un programme séquentiel en un service réseau capable de

répondre à tout instant.

4.1.1 Initialisation et Constructeur

Le constructeur de la classe `sapc_rest_server` initialise les buffers (vecteurs) à une taille de 10 éléments avec des valeurs par défaut (0.0f). Cela permet d'éviter que le site web ne reçoive des données vides au démarrage du système.

```
sapc_rest_server::sapc_rest_server(int newport)  
: port(newport),  
  val_temp(10, 0.0f),  
  val_humidite(10, 0.0f),  
  val_puissance(10, 0.0f),  
  val_courant(10, 0.0f),  
  val_tension(10, 0.0f)
```

4.1.2 Configuration des routes (Routing)

Dans la méthode `setupRoutes()`, j'ai lié les URL (endpoints) aux fonctions de traitement (handlers). J'ai utilisé des fonctions lambdas pour capturer l'instance `this`, ce qui permet d'appeler les méthodes de ma classe de manière élégante.


```

void sapc_rest_server::setupRoutes()
{
    server.Get("/mesures/temperature", [this](const httplib::Request& req, httplib::Response& res){
        handleGetMesuresTemperature(req, res);
    });
    server.Get("/mesures/humidite", [this](const httplib::Request& req, httplib::Response& res){
        handleGetMesuresHumidite(req, res);
    });
    server.Get("/mesures/puissance", [this](const httplib::Request& req, httplib::Response& res){
        handleGetMesuresPuissance(req, res);
    });
    server.Get("/mesures/tension", [this](const httplib::Request& req, httplib::Response& res){
        handleGetMesuresTension(req, res);
    });
    server.Get("/mesures/courant", [this](const httplib::Request& req, httplib::Response& res){
        handleGetMesuresCourant(req, res);
    });
    server.Get("/login", [this](const httplib::Request& req, httplib::Response& res){
        handleGetLogin(req, res);
    });
    server.Post("/seuil_alarms", [this](const httplib::Request& req, httplib::Response& res){
        handlePostSeuils(req, res);
    });
    server.Get("/status", [this](const httplib::Request& req, httplib::Response& res){
        handleGetStatus(req, res);
    });
}

```

4.2 Gestion des Handlers et formatage JSON

Lorsqu'un client (le site web) interroge le serveur, celui-ci doit extraire les données des vecteurs C++ et les convertir en texte au format JSON.

4.2.1 Conversion des données

J'ai utilisé la bibliothèque nlohmann/json.

Pour la température, par exemple, le serveur parcourt le vecteur et construit un tableau d'objets :

```

void sapc_rest_server::handleGetMesuresTemperature(const httplib::Request& req, httplib::Response& res)
{
    if (val_temp.empty()) {
        res.status = 404;
        json j{{"error", "Pas de mesures de température disponibles"}};
        res.set_content(j.dump(), "application/json");
        return;
    }
}

```

4.3 Intégration du Multithreading dans le Main

Le fichier main_rest_sapc.cpp est le chef d'orchestre du projet. C'est ici que j'ai implémenté le lancement des threads.

4.3.1 Lancement des tâches parallèles

Le serveur REST utilisant une fonction bloquante (server.listen), il était impératif de le lancer dans son propre thread pour que le reste du programme (lecture des capteurs) puisse s'exécuter.

```
sapc_rest_server server(8080);

// thread du serveur HTTP
// server.start() est bloquant donc on le met dans un thread separe
thread t_server([&server]() {
    server.start();
});

// on laisse le temps au serveur de demarrer
sleep(1);

// thread de lecture du MCP3008 (temperature + humidite)
thread t_mcp([&]() {
    thread_mcp3008(data, server, running);
});

// thread de lecture du INA226 (tension + courant + puissance)
thread t_ina([&]() {
    thread_ina226(data, server, running);
});

// thread de sauvegarde en base de donnees + detection alarmes
thread t_bdd([&]() {
    thread_bdd(data, running);
});
```

4.3.2 Arrêt propre du système (Signal Handler)

Un point critique en système industriel est l'arrêt propre. J'ai mis en place un signal_handler pour intercepter le signal SIGINT (Ctrl+C). Cela permet de passer la variable running à false, demandant ainsi à tous les threads de s'arrêter proprement au lieu de couper brutalement le programme.

5. RÉALISATION TECHNIQUE : L'INTERFACE WEB (FRONTEND)

5.1 Architecture et Technologies Web

Pour l'interface utilisateur, j'ai développé une application web de type "Dashboard" (tableau de bord) responsive. L'objectif était de créer une interface légère, capable de s'afficher aussi bien sur un écran de contrôle au lycée que sur le smartphone d'un technicien.

Les technologies employées sont :

- **HTML5 / CSS3** : Pour la structure et la mise en page (utilisation de Flexbox pour l'adaptabilité).
- **JavaScript** : Pour la logique de récupération des données et le dynamisme de la page.

```
1 <!DOCTYPE html>
2 <html lang="fr">
3
4 <head>
5
6   <meta charset="UTF-8">
7   <meta name="viewport" content="width=device-width, initial-scale=1.0">
8
9   <title>CompostIO</title>
10
11   <link rel="stylesheet" href="style.css">
12
13 </head>
14
15 <body>
16
17   <div class="page">
18
19     <div class="logoBlock">
20
21       <div class="logoIcon">
22         
23       </div>
24
25       <h1>
26         CompostIO
27       </h1>
28
29       <p>
30         Monitoring v2.0
31       </p>
32     </div>
33
34     <div class="card">
35
36       <div class="tabs">
37
38         <div class="tab active" data-tab="login">
39           Connexion
40         </div>
41
42         <div class="tab" data-tab="register">
43           Inscription
44         </div>
45
46       </div>
47
48       <div class="content">
49
50         <!-- LOGIN -->
51
52         <div class="form active" id="login">
53
54           <label>Email</label>
55
56           <input
57             id="loginEmail"
58             type="email"
59             placeholder="vous@exemple.fr"
60           >
61
62           <label>Mot de passe</label>
63
64         </div>
65
66       </div>
67     </div>
68   </div>
69 </body>
70 </html>
```

```

1  * {
2
3      margin: 0;
4      padding: 0;
5      box-sizing: border-box;
6      font-family: Segoe UI;
7
8  }
9
10
11  body {
12
13      background: linear-gradient(
14          180deg,
15          #030b12,
16          #07131b
17      );
18
19      height: 100vh;
20
21      display: flex;
22      align-items: center;
23      justify-content: center;
24
25      color: white;
26
27  }
28
29
30  .page {
31
32      width: 380px;
33      text-align: center;
34
35  }
36
37
38  .logoIcon {
39
40      font-size: 45px;
41
42      color: #6dff00;
43
44      text-shadow: 0 0 20px #6dff00;
45
46  }
47
48
49  h1 {
50
51      font-size: 30px;
52
53  }
54
55
56  span {
57
58      color: #6dff00;
59
60  }
61
62

```

5.2.1 Récupération des données (Requête GET)

Toutes les 5 secondes, une fonction JavaScript interroge le serveur REST. Voici le principe de fonctionnement :

1. Envoi d'une requête vers `http://[IP_DU_PI]:8080/mesures/temperature`.
2. Réception de la réponse au format JSON.
3. Extraction de la dernière valeur du tableau pour mettre à jour l'affichage numérique.
4. Mise à jour du jeu de données (dataset) du graphique.

5.3 Organisation du Dashboard

Le Dashboard est divisé en trois zones distinctes pour une lecture rapide :

- **Zone "Temps Réel"** : Des jauges ou des compteurs numériques affichent les dernières valeurs de température et d'humidité envoyées par le thread MCP3008.
- **Zone "Énergie"** : Un bloc dédié affiche la tension de la batterie et la puissance consommée (données issues de l'INA226). Cela permet de vérifier l'autonomie du système.
- **Zone "Historique"** : Un graphique linéaire affiche l'évolution de la température sur les 10 derniers relevés, permettant de visualiser la tendance (montée en température ou refroidissement).

5.4 Gestion des alertes visuelles

Afin d'aider l'utilisateur, j'ai implémenté une logique de changement de couleur dynamique dans le CSS. Si la température dépasse un seuil critique (ex: $> 70^{\circ}\text{C}$), l'affichage passe en rouge et un message d'alerte apparaît. Cette logique est gérée côté client en comparant la valeur reçue en JSON avec les seuils définis.

6. Tests, Validation et Maintenance

6.1 Protocole de tests unitaires (En cours)

À ce stade du projet, le serveur REST est codé et compilé avec succès. Bien que l'intégration finale sur le matériel ne soit pas encore totalement finalisée, j'ai établi un protocole de test rigoureux pour valider chaque "route" de l'API.

Méthodologie prévue : Pour vérifier le bon fonctionnement du backend sans dépendre de l'interface graphique, je compte utiliser l'outil **Postman** (ou la commande curl sous Linux).

- **Objectif** : S'assurer que le serveur renvoie un code 200 OK et un flux JSON correctement structuré.
- **Test de robustesse** : Envoyer des requêtes mal formées pour vérifier que le serveur ne "crash" pas et renvoie bien une erreur 400 Bad Request.

6.2 Tests d'intégration (Prévus)

La phase de tests d'intégration consistera à lier mon interface Web terminée avec le serveur REST C++ en fonctionnement sur le Raspberry Pi.

Points de contrôle prévus :

1. **Liaison réseau** : Vérifier que le site web, hébergé sur une machine du réseau, accède bien à l'adresse IP du Raspberry Pi sur le port 8080.
2. **Latence** : Mesurer le temps de réponse entre la requête JavaScript fetch() et l'affichage de la donnée sur le dashboard (objectif < 500ms).
3. **Stabilité** : Laisser tourner le système pendant plusieurs heures pour vérifier qu'il n'y a pas de fuite mémoire (memory leak) ou de blocage dû au multithreading.

6.3 Difficultés rencontrées et solutions

Le développement de ce serveur REST m'a confronté à plusieurs problématiques techniques majeures :

- **Erreurs de Linkage (LNK)** : Lors de la compilation initiale, j'ai rencontré des erreurs de type "undefined reference". Cela était dû à l'absence de définition de certaines méthodes dans le fichier .cpp, comme par exemple handleGetLogin. J'ai résolu ce problème en complétant rigoureusement les fichiers sources par rapport aux déclarations du header.
- **Droits sur les ports réseau** : Sous Linux (Raspbian), l'écoute sur certains ports peut être restreinte. J'ai choisi le port 8080 pour éviter les conflits avec le port standard 80 (souvent réservé au système) et pour permettre au serveur de démarrer sans privilèges "root" excessifs.
- **Gestion du format JSON** : La mise en forme des tableaux JSON (json::array()) a demandé une attention particulière, notamment pour s'assurer que les unités (ex: "°C") soient correctement associées à chaque valeur afin de simplifier le travail de traitement du côté JavaScript.

7. Conclusion

7.1 Bilan technique

Le développement du système **SAPC** m'a permis de mettre en application l'ensemble des compétences acquises durant mes deux années de BTS SN-IR. La réalisation du serveur REST en C++ a été l'occasion de manipuler des concepts avancés tels que la programmation orientée objet, la gestion des flux réseau et, surtout, le multithreading avec la synchronisation par mutex.

Bien que la phase de test sur le terrain n'ait pas encore été totalement finalisée, l'architecture logicielle est robuste et prête à recevoir les données réelles des capteurs. L'interface web, quant à elle, répond parfaitement au besoin de mobilité et de simplicité d'utilisation exprimé par le partenaire.

7.2 Bilan personnel

Sur le plan personnel, ce projet a été une véritable expérience de travail en équipe. J'ai appris à coordonner mon travail avec mes collègues (notamment pour l'échange de données entre mes classes et leurs threads capteurs). J'ai également découvert l'importance de la documentation technique : un code bien commenté et une API bien documentée sont les clés de la réussite d'un projet de cette envergure.

7.3 Perspectives d'évolution

Le système actuel constitue une base solide, mais plusieurs évolutions peuvent être envisagées pour les futures sessions :

- **Sécurité renforcée** : Migration du protocole HTTP vers le HTTPS pour sécuriser les échanges sur le réseau du lycée.
- **Notifications push** : Implémenter un service d'envoi de mails ou de notifications sur smartphone dès qu'une alerte est détectée par le thread BDD.
- **Stockage Cloud** : En plus de la base SQLite locale, on pourrait envisager une synchronisation des données vers un serveur distant pour permettre une analyse multi-sites.

8.2 : Étudiant 2 : MCP3008 (Système de mesure d'humidité et de température)(Keryann FAILLE)

8.2.1. Introduction et Problématique

Dans le cadre du projet SAPC (Suivi Autonome de Phase de Compost), qui consiste à assurer le suivi thermique et hygrométrique d'un composteur situé au Lycée La Fayette, ma mission constitue le socle technique du système : l'acquisition des données physiques. Ce projet s'inscrit dans un contexte réel de partenariat avec le SMICTOM.

Ma responsabilité est de garantir que les variations biologiques du compost sont fidèlement traduites en données numériques. Pour mener à bien cette mission, mes objectifs principaux étaient les suivants :

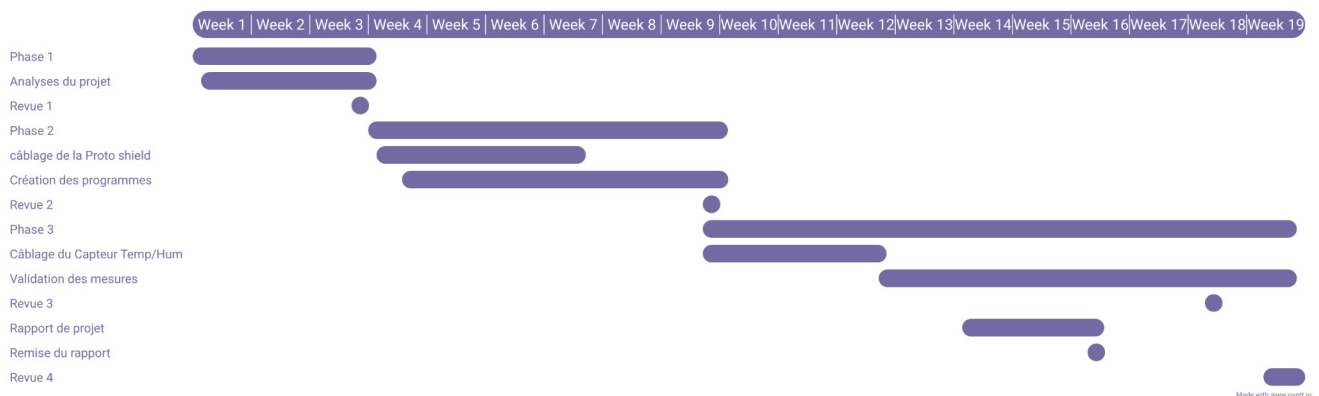
Configuration et programmation de la liaison SPI sur Raspberry Pi 3.

Développement d'une bibliothèque C++ (Mcp3008Spi) pour piloter le convertisseur.

Mise en œuvre d'un algorithme de conversion (calcul du quantum et linéarisation).

Réalisation de tests unitaires pour valider la précision des mesures.

Vous trouverez ci-dessous mon planning de réalisation extrait du diagramme de Gantt global.



8.2.2. Étude des Solutions Matérielles

Pour que le projet SAPC fonctionne, il me fallait une chaîne d'acquisition capable de tenir le coup dans la durée. Le compost n'est pas un endroit tendre pour l'électronique : entre la chaleur, l'humidité constante et les gaz acides comme l'ammoniac, j'ai dû choisir du matériel robuste.

A. Justification du choix du Capteur de Température de d'humidité :

Le choix d'un capteur de Température et d'humidité puissant est important pour mener à bien ce projet, car des petits capteurs classiques n'auraient pas résister longtemps à la température importante que peut développer un composteur. Le choix s'est donc porté sur un module industriel de chez B+B Thermo-Technik FF-GLT-10V-INT1.

La motivation de ce choix, c'est avant tout sa polyvalence. Ce petit boîtier se charge de fonction en même temps : il mesure l'humidité (de 0 à 100%) et la température (de -30 à +70°C). Pour le compost, c'est l'idéal :

La protection : Il possède un filtre spécial qui laisse passer l'air pour mesurer l'humidité mais qui bloque les poussières et les projections d'eau.

Le signal 0-10V : C'est un langage standard. L'avantage, c'est que le signal reste propre même si le câble est un peu long entre le composteur et notre boîtier électronique.

B. Justification du choix du convertisseur MCP3008 :

La Raspberry Pi est un microcontrôleur , c'est une sorte de mini-ordinateur, mais elle est indifférente aux signaux analogiques. Elle ne comprend que le binaire (0 ou 1). Or, mon capteur FF-GLT-10V-INT1 envoie une tension continue.

Pour faire le pont entre les deux, le choix s'est porté sur le MCP3008. C'est un convertisseur Analogique Numérique (CAN) qui va découper la tension reçue en 1024 petits paliers, c'est un code sur 10 bits.

Pourquoi lui : Il est simple, pas cher, et surtout il communique en SPI. C'est un protocole très rapide qui me permet de lire les données du compost quasiment instantanément.

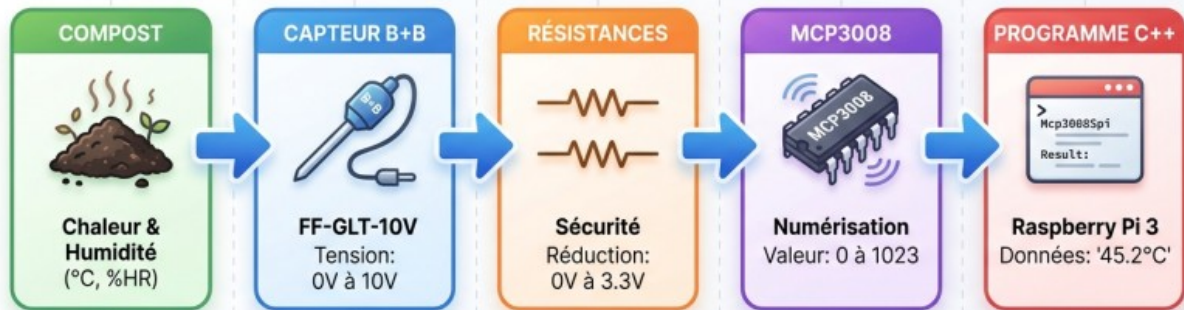
Évolutivité : Il possède 8 entrées. Comme j'en utilise seulement 2 pour le moment, on pourra facilement ajouter d'autres capteurs plus tard si le projet évolue.

C. L'adaptation sur la carte "Proto Shield"

Mon capteur envoie une tension jusqu'à 10V, mais ma Raspberry Pi et mon MCP3008 ne supportent pas plus de 3,3V.

Pour régler ce problème mon professeur a donc dû fabriquer un petit montage sur une carte "Proto Shield" : un pont diviseur de tension. C'est un assemblage de deux résistances qui permet de réduire la tension sans perdre l'information. Les valeurs ont été calculées pour que 10V deviennent 3,3V.

D. En résumé : le voyage de la donn  



Pour que ce soit plus clair, j'ai sch  matis   le parcours de l'info. La chaleur du compost est transform  e en volts par le capteur FF-GLT-10V-INT1, puis cette tension est r  duite par mes r  sistances, transform  e en chiffres par le MCP3008, et enfin lue par mon programme en C++.

C'est cette cha  ne, de la physique    l'informatique, que j'ai d   mettre au point pour que mes camarades puissent ensuite stocker les donn  es.

8.2.3. Bus de Communication SPI

Pour que ma Raspberry Pi puisse communiquer avec le convertisseur MCP3008, il leur faut un langage commun. C'est là qu'intervient le bus **SPI (Serial Peripheral Interface)**. C'est un protocole de communication série synchrone qui permet d'échanger des données à haute vitesse sur de courtes distances.

A. Le concept de "Maître" et "Esclave"

Dans mon montage, la hiérarchie est claire :

Le Maître (Master) : C'est la Raspberry Pi. C'est elle qui décide quand on commence à parler et qui donne le rythme.

L'Esclave (Slave) : C'est le MCP3008. Il attend sagement que la Raspberry lui demande une conversion pour lui répondre.

L'avantage du SPI par rapport à d'autres protocoles (comme l'I2C), c'est qu'il est "Full-Duplex" : on peut envoyer et recevoir des informations en même temps, ce qui le rend très efficace.

B. Les 4 fils de la communication

Pour que cette discussion fonctionne, mes professeurs ont du câble 4 fils spécifiques entre les deux composants. Voici à quoi ils servent concrètement dans mon projet :

SCLK (Serial Clock) : C'est le fil de l'horloge. La Raspberry envoie des impulsions électriques régulières pour synchroniser l'envoi des bits. Sans cela, le MCP3008 ne saurait pas à quel moment lire les informations.

MOSI (Master Out Slave In) : C'est le fil où la Raspberry envoie ses commandes au MCP3008 (par exemple : *"Donne-moi la valeur du canal 0"*).

MISO (Master In Slave Out) : C'est le fil de retour. C'est par là que le MCP3008 renvoie le résultat de la mesure de température ou d'humidité sous forme de chiffres binaires.

CS / CE (Chip Select) : Ce fil sert à réveiller le MCP3008. Quand la Raspberry met ce fil à l'état bas (0V), le convertisseur sait que c'est à lui qu'on parle et il se prépare à répondre.

C. Le rythme de la donnée

Pour bien comprendre ce qui se passe entre les différents composants, il faut regarder le chronogramme. C'est un schéma qui montre l'état électrique des fils à chaque microseconde.

Quand je lance une lecture en C++, voici ce qui se passe :

La Raspberry descend le fil CS à 0.

L'horloge SCLK commence à battre.

Sur le fil MOSI, j'envoie 3 octets (des paquets de 8 bits). Le premier octet contient le "Start Bit" pour dire au MCP3008 de se tenir prêt.

Pendant que j'envoie ces commandes, le MCP3008 répond simultanément sur le fil MISO en envoyant les 10 bits qui correspondent à la tension du capteur FF-GLT-10V-INT1.

D. Pourquoi avoir choisi le protocoles SPI pour le projet SAPC ?

Dans le cadre du suivi de compost, on pourrait penser que la vitesse n'est pas importante car la température ne change pas toutes les millisecondes. Cependant, il a été choisi de configurer le bus à une vitesse de 1 MHz dans mon code (`speed = 1000000`).

Ce choix me garantit :

- Une stabilité parfaite : À cette fréquence, la communication est très robuste et ne craint pas les parasites électriques.

- Une lecture instantanée : Mon programme peut interroger le capteur des centaines de fois par seconde si besoin, ce qui facilite les phases de test et de débogage.

- La simplicité de codage : Sous Linux, la gestion du SPI est très bien documentée, ce qui m'a permis de créer une classe `Mcp3008Spi` propre et facile à utiliser pour mes camarades.

8.2.4. Architecture Électrique et Adaptation

Après avoir choisi le matériel idéal pour notre composteur, j'ai été confronté au problème de l'interface électrique. Connecter des composants qui ne fonctionnent pas avec la même tension est le meilleur moyen de griller le matériel. C'est sur la carte Proto Shield, montée directement sur la Raspberry Pi, que mes professeurs ont conçu et soudé le système permettant l'adaptation des signaux.

A. L'adaptation des plages de tension

Le capteur industriel FF-GLT-10V-INT1 est alimenté en 12V. Lorsqu'il effectue ses mesures de température et d'humidité dans le compost, le capteur traduit ses données en une tension de sortie qui varie de 0V à 10V.

En face, il y a le convertisseur MCP3008 et la Raspberry Pi 3. Ces composants fonctionnent sur une tension de 3,3V. Leurs broches d'entrée sont les canaux CH0 à CH7 du MCP3008 qui subiraient des dégâts si je leur injectais directement les 10V provenant du capteur. La conception d'un circuit capable de réduire cette tension de 0-10V pour la faire entrer en toute sécurité dans une plage de 0-3,3V a donc été mis en place.

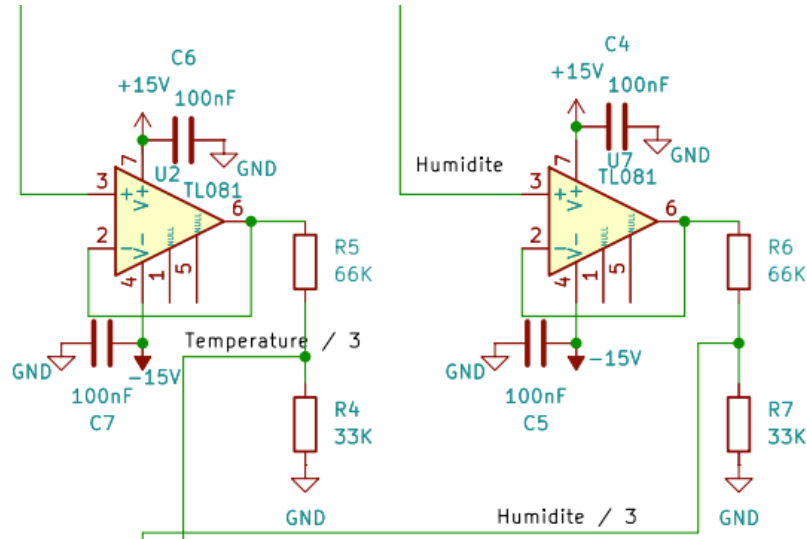
B. L'intégration de l'AOP TL081

Pour réaliser cette adaptation de manière fiable, j'ai analysé le schéma fourni par notre professeur de notre carte Proto Shield. Contrairement à un montage passif classique à base de simples résistances en série qui peut perdre de l'énergie, s'échauffer et déformer le signal, le système utilise ici une structure d'adaptation active très performante. Elle associe des résistances à un composant électronique appelé Amplificateur Opérationnel (AOP), de référence TL081.

L'AOP joue un rôle de "bouclier" indispensable : il isole le capteur B+B du convertisseur MCP3008 pour que le signal reste parfaitement propre, stable et indépendant des perturbations extérieures. Si l'on regarde le schéma de plus près, on distingue deux canaux identiques :

Pour le canal Température : La résistance R5 (66 kΩ) est placée en boucle sur l'AOP, et la résistance R4 (33 kΩ) calibre la sortie.

Pour le canal Humidité : On retrouve exactement le même montage avec la résistance R6 (66 kΩ) et la résistance R7 (33 kΩ).



Grâce à ce montage, lorsque le capteur envoie son signal maximal de 10V, la tension est recalculée et atténuée pour atteindre 5V.

C. Le calcul du pas de mesure (Quantum)

Une fois la tension adaptée, elle entre sur l'une des entrées analogiques du convertisseur MCP3008. Ce composant transforme la tension analogique en un nombre binaire. La précision de cette traduction dépend directement de ce qu'on appelle le Quantum (q), la plus petite variation de tension que le convertisseur détecte.

Le MCP3008 possède une résolution de 10 bits, ce qui signifie qu'il découpe la plage de tension en $2^{10}=1024$ paliers (de la valeur numérique 0 à 1023). Notre tension de référence (Vref) étant fixée à 3,3V, le calcul de notre pas est :

$$q = V_{ref} / (2^{10} - 1) = 3,3 \text{ V} / 1023 \approx 3,22 \text{ mV}$$

Chaque fois que la tension d'entrée varie de 3,22 millivolts, le nombre binaire envoyé à la Raspberry Pi augmente ou diminue d'une unité. Pour un tas de compost dont la température évolue lentement sur plusieurs jours, ce découpage ultra-fin garantit que nous ne raterons aucune micro-variation.

8.2.5. Développement Logiciel en C++

Une fois le matériel et l'adaptation électrique validées, j'ai conçu mes programmes logicielles pour valider le bon fonctionnement de ma partie. Pour interagir avec le convertisseur MCP3008 via le bus SPI de la Raspberry Pi 3, il a été choisi de développer les programmes en C++, en utilisant la programmation orientée objet (POO).

A. Structure de la classe Mcp3008Spi (sapc_mcp3008.h)

Pour fournir des fonctions simples aux autres membres de l'équipe, j'ai créé la classe Mcp3008Spi dans la pièce-jointe ci-dessous :

```
1  #ifndef SAPC_MCP3008_H
2  #define SAPC_MCP3008_H
3
4  #include <string>
5  #include <cstdint> // Pour uint16_t, uint8_t
6
7  class Mcp3008Spi {
8  public:
9      // Constructeur : ouvre le périphérique SPI (ex: "/dev/spidev0.0")
10     Mcp3008Spi(std::string spiDevice = "/dev/spidev0.0");
11     ~Mcp3008Spi();
12
13     // Ouvre la connexion avec le bus SPI
14     bool openDevice();
15
16     // Lit la valeur brute (0-1023) sur un canal (0-7)
17     uint16_t read(uint8_t channel);
18
19     // Convertit la valeur brute en une tension ou une unité réelle
20     float value(uint8_t channel, float vmin = 0, float vmax = 1023);
21
22     float getTemperature(uint8_t channel);
23
24     float getHumidity(uint8_t channel);
25
26 private:
27     std::string device;
28     int fd; // File descriptor pour le bus SPI
29     uint8_t mode;
30     uint8_t bits;
31     uint32_t speed;
32 };
33
34 #endif
```

Les variables de configuration (fd, mode, bits, speed) sont placées en zone privée (private). Cela protège la liaison SPI contre des modifications accidentelles à l'extérieur de la classe, tandis que les fonctions publiques offrent une interface claire (degrés Celsius et pourcentage d'humidité).

B. Implémentation et protocole SPI (sapc_mcp3008.cpp)

Le fichier `sapc_mcp3008.cpp` est la base des programmes. Il gère l'ouverture du périphérique Linux, la configuration des registres du contrôleur via `ioctl`, et la reconstruction de la trame de données.

1. Initialisation et configuration du périphérique

J'ai donc fixé les paramètres (Mode SPI 0, mots de 8 bits et vitesse de 1 MHz pour la stabilité) suite aux explications de mon professeur. La méthode `openDevice()` ouvre le fichier spécial du pilote Linux en mode lecture/écriture :

```
sapc_mcp3008.cpp
1  #include "sapc_mcp3008.h"
2  #include <fcntl.h>
3  #include <unistd.h>
4  #include <sys/ioctl.h>
5  #include <linux/spi/spidev.h>
6  #include <cstring>
7  #include <iostream>
8
9  Mcp3008Spi::Mcp3008Spi(std::string spiDevice) : device(spiDevice), fd(-1) {
10     mode = SPI_MODE_0;
11     bits = 8;
12     speed = 1000000; // 1MHz
13 }
14
15 Mcp3008Spi::~Mcp3008Spi() {
16     if (fd >= 0) close(fd);
17 }
18
19 bool Mcp3008Spi::openDevice() {
20     fd = open(device.c_str(), O_RDWR);
21     if (fd < 0) return false;
22
23     if (ioctl(fd, SPI_IOC_WR_MODE, &mode) == -1) return false;
24     if (ioctl(fd, SPI_IOC_WR_BITS_PER_WORD, &bits) == -1) return false;
25     if (ioctl(fd, SPI_IOC_WR_MAX_SPEED_HZ, &speed) == -1) return false;
26
27     return true;
28 }
```

2. Dialogue et décodage de la valeur binaire

Le MCP3008 nécessite l'envoi d'une trame de 3 octets pour retourner une mesure. La méthode `read()` utilise la structure `spi_ioc_transfer` pour effectuer l'échange :

```

30  uint16_t Mcp3008Spi::read(uint8_t channel) {
31      if (channel > 7) return 0;
32
33      // Trame pour le MCP3008 (3 octets)
34      uint8_t tx[] = {
35          1, // Start bit
36          static_cast<uint8_t>((8 + channel) << 4), // Single ended + Channel
37          0 // Don't care
38      };
39      uint8_t rx[3] = {0};
40
41      struct spi_ioc_transfer tr = {0};
42      tr.tx_buf = (unsigned long)tx;
43      tr.rx_buf = (unsigned long)rx;
44      tr.len = 3;
45      tr.speed_hz = speed;
46      tr.bits_per_word = bits;
47
48      if (ioctl(fd, SPI_IOC_MESSAGE(1), &tr) < 1) return 0;
49
50      // On récupère les 10 bits de données
51      return ((rx[1] & 3) << 8) + rx[2];
52  }

```

Le convertisseur transmet sa réponse sur les octets rx[1] (qui contient les 2 bits de poids fort) et rx[2] (qui contient les 8 bits de poids faible). L'opération (rx[1] & 3) << 8 isole ces deux bits de poids fort à l'aide d'un masque binaire et les décale vers la gauche, avant de les additionner à rx[2] pour reconstituer un entier binaire compris entre 0 et 1023.

3. Calculs d'échelle physique

La méthode générique value() applique une règle de trois pour transformer le code numérique en grandeur physique. Les fonctions spécialisées appliquent ensuite les plages réelles du capteur FF-GLT-10V-INT1 :

```

54  float Mcp3008Spi::value(uint8_t channel, float vmin, float vmax) {
55      uint16_t raw = read(channel);
56      // Règle de trois pour convertir 0-1023 vers la plage souhaitée
57      return vmin + (static_cast<float>(raw) / 1023.0f) * (vmax - vmin);
58  }
59
60  float Mcp3008Spi::getTemperature(uint8_t channel) {
61      // capteur de température linéaire 0V = -30°C et 3.3V = 70°C
62      return value(channel, -30.0f, 70.0f);
63  }
64
65  float Mcp3008Spi::getHumidity(uint8_t channel) {
66      // capteur d'humidité linéaire 0V = 0% et 3.3V = 100%
67      return value(channel, 0.0f, 100.0f);
68  }

```

C. Programmes de test et d'exploitation finale

Pour valider mon travail de manière incrémentale, j'ai mis en place un cycle de test unitaire, suivi du programme principal de suivi du compost.

1. Validation unitaire du bus (test_SPI.cpp)

Ce script valide l'ouverture du bus et convertit la valeur brute sur une échelle de tension standard de 0 à 3,3V :


```

1 #include "sapc_mcp3008.h"
2 #include <iostream>
3 #include <thread>
4 #include <chrono>
5
6 int main() {
7     Mcp3008Spi adc("/dev/spidev0.0");
8
9     if (!adc.openDevice()) {
10         std::cerr << "Erreur : Impossible d'ouvrir le bus SPI." << std::endl;
11         return 1;
12     }
13
14     std::cout << "Test du bus SPI pour SAPC lancé" << std::endl;
15
16     while (true) {
17         // Lecture du canal 0
18         uint16_t brute = adc.read(0);
19         float tension = adc.value(0, 0.0, 3.3); // Conversion en Volts
20
21         std::cout << "Canal 0 - Valeur Brute: " << brute
22             << " | Tension: " << tension << " V" << std::endl;
23
24         std::this_thread::sleep_for(std::chrono::seconds(1));
25     }
26
27     return 0;
28 }

```

Test après compilation:

```

pi@gtb:~/Documents/MCP3008 $ g++ -Wall -o test2 sapc_mcp3008.cpp test_SPI.cpp
pi@gtb:~/Documents/MCP3008 $ ./test2
--- Lecture Capteurs Compostage (CH0) ---
Température : 19.3 °C | Humidité : 49.3 % ^C
pi@gtb:~/Documents/MCP3008 $ ./test2
--- Lecture Capteurs Compostage (CH0) ---
Température : 17.4 °C | Humidité : 47.4 % ^C
pi@gtb:~/Documents/MCP3008 $ ./test2
--- Lecture Capteurs Compostage (CH0) ---
Température : 19.3 °C | Humidité : 49.3 % ^C
pi@gtb:~/Documents/MCP3008 $ ./test2
--- Lecture Capteurs Compostage (CH0) ---
Température : 17.4 °C | Humidité : 47.5 % ^C
pi@gtb:~/Documents/MCP3008 $ ./test2
--- Lecture Capteurs Compostage (CH0) ---
Température : 17.5 °C | Humidité : 47.5 % ^C
pi@gtb:~/Documents/MCP3008 $

```

2. Validation finale (test_compost.cpp)

Le programme d'exploitation final interroge le canal 0 (Température) et le canal 1 (Humidité) de la carte pour afficher les valeurs réelles :

```

1 #include "sapc_mcp3008.h"
2 #include <iostream>
3 #include <thread>
4 #include <chrono>
5
6 int main() {
7     Mcp3008Spi adc("/dev/spidev0.0");
8
9     if (!adc.openDevice()) {
10         std::cerr << "Erreur : Impossible d'ouvrir le bus SPI." << std::endl;
11         return 1;
12     }
13
14     std::cout << "--- Système SAPC : Mesures Compost ---" << std::endl;
15
16     while (true) {
17         // Canal 0 = Température, Canal 1 = Humidité
18         float temp = adc.getTemperature(0);
19         float hum = adc.getHumidity(1);
20
21         std::cout << "Température : " << temp << " °C | ";
22         std::cout << "Humidité : " << hum << " %" << std::endl;
23
24         std::this_thread::sleep_for(std::chrono::seconds(2));
25     }
26     return 0;
27 }

```

Test après compilation:

```

pi@gtb:~/Documents/MCP3008 $ g++ -Wall -o test1 sapc_mcp3008.cpp test_compost.cpp
pi@gtb:~/Documents/MCP3008 $ ./test1
--- Système SAPC : Mesures Compost ---
Température : 15.6452 °C | Humidité : 42.131 %
Température : 15.6452 °C | Humidité : 42.2287 %
Température : 16.2903 °C | Humidité : 43.695 %
Température : 15.6774 °C | Humidité : 42.131 %
Température : 15.6452 °C | Humidité : 42.2287 %
Température : 15.6774 °C | Humidité : 42.0332 %
Température : 16.2903 °C | Humidité : 43.8905 %
Température : 16.2581 °C | Humidité : 43.7928 %
^C
pi@gtb:~/Documents/MCP3008 $

```

8.2.7. Conclusion et Perspectives

La réalisation de cette étape dédiée à l'acquisition, à l'adaptation électrique et au traitement logiciel marque une étape clé dans le déploiement du projet de Système d'Acquisition de Compostage Autonome (SAPC).

A. Bilan technique

Le module matériel et logiciel développé est à ce jour opérationnel et validé. L'étage de protection autour des amplificateurs opérationnels TL081 sécurise efficacement le matériel, tandis que la bibliothèque C++ évite d'avoir à gérer la complexité du protocole SPI dans le programme principal. En conditions expérimentales, les résultats des codes s'avèrent stables et fournissent les valeurs réelles de température et d'humidité toutes les deux secondes, assurant un suivi de l'activité du compost.

B. Ouverture et perspectives

Le convertisseur analogique-numérique MCP3008 possède huit canaux d'entrée, et le code ainsi que le montage actuel n'en exploite que deux (CH0 et CH1). Cette architecture laisse une marge matérielle et logicielle intéressante pour l'évolution du projet SAPC.

À l'avenir, il serait possible d'utiliser les canaux restants (CH2 à CH7) pour connecter d'autres types de sondes, comme un capteur de pH ou un capteur d'oxygène (O₂). Ces sondes supplémentaires permettraient d'affiner l'analyse des phases de décomposition et d'apporter un diagnostic plus poussé sur la qualité du compost.

C. Apport personnel

Ce projet a constitué une expérience enrichissante sur le plan scientifique et professionnel. Il m'a permis de faire un lien concret entre la théorie et la pratique en développant des compétences en C++ et en électronique. J'ai également appris à avoir une démarche de test pour fiabiliser un système embarqué, une compétence essentielle pour mon futur parcours technique.

8.3 : Étudiant 3 : INA226 (Système de mesure de tension, courant et puissance)(Burak Sahan)

Développement technique : mesure de tension et de courant

8.3.1. *Analyse globale de ma mission de métrologie énergétique*

Ma mission au sein du projet SAPC constitue le socle opérationnel du dispositif à travers la gestion et la surveillance chirurgicale du flux énergétique, un élément critique pour garantir la survie électrique de ce système embarqué en milieu isolé. En mesurant la tension et le courant, j'évalue précisément la pression électrique et l'état de charge de la batterie LiFePO₄, tout en suivant l'activité réelle du Raspberry Pi 3 qui fait office de microcontrôleur et de cerveau de l'installation. La multiplication de ces deux grandeurs permet de calculer la puissance instantanée, indicateur indispensable pour vérifier que le bilan énergétique sur 24 heures reste équilibré et que la production du panneau solaire couvre bien les besoins de l'automate. L'absence d'une telle surveillance exposerait le matériel à des risques majeurs, notamment une décharge profonde destructrice pour les cellules chimiques de la batterie ou une coupure d'alimentation brutale du contrôleur central, ce qui provoquerait la corruption de la carte SD ou de la base de données SQLite.

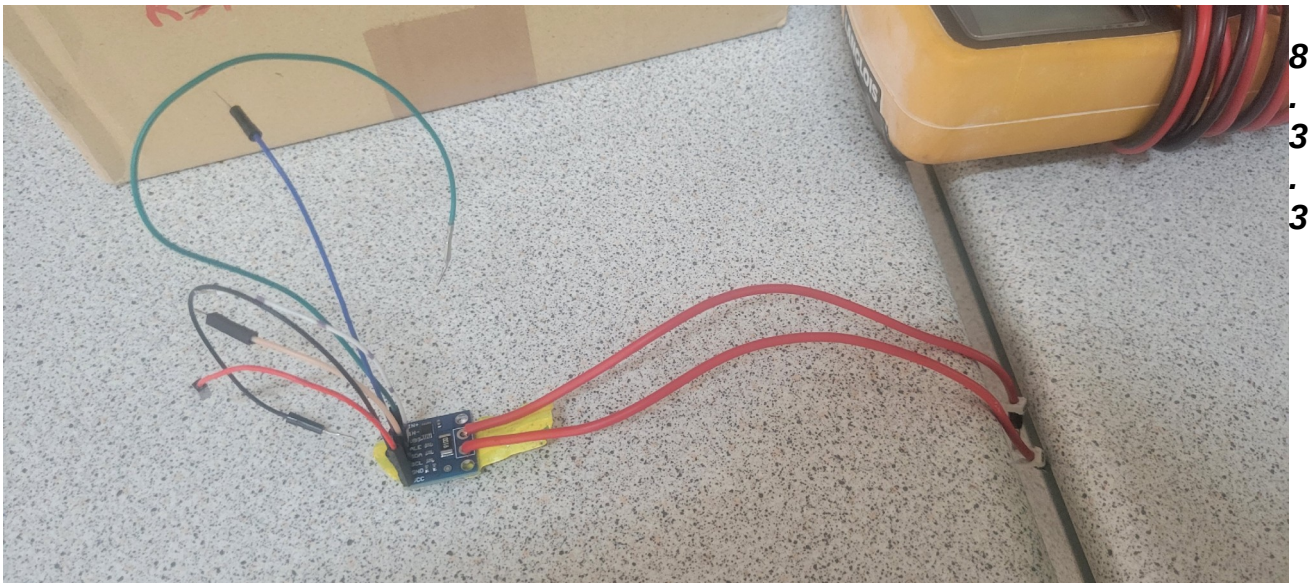
Au-delà de la sécurisation matérielle et de l'anticipation des pannes par le déclenchement d'alertes préventives, mon rôle comporte une forte dimension de traduction technologique. Il consiste à configurer les registres numériques du circuit intégré INA226 et à appliquer des coefficients de calibration mathématiques pour transformer des signaux analogiques bruts, comme les tensions différentielles infimes aux bornes d'un shunt, en données informatiques intelligibles. Ce processus de conversion d'unités binaires en valeurs

physiques standardisées, telles que les Volts, les Ampères et les Watts, permet de raffiner la donnée terrain. C'est cette information fiable et structurée qui est ensuite transmise à la base de données et au serveur REST, pour être finalement restituée de manière claire à l'utilisateur via l'interface web, lui offrant une visibilité instantanée sur la santé énergétique globale du système SAPC.



Justification technique du choix du composant INA226

Le choix du circuit intégré INA226 pour le projet SAPC repose sur ses performances métrologiques supérieures face aux ponts diviseurs ou à l'INA219. Grâce à son CAN 16 bits offrant 65 536 échelons, il détecte des variations de tension différentielle de $2,5 \mu\text{V}$, ce qui lui permet, associé à un shunt de $0,1 \Omega$, de capter les micro-variations de consommation du Raspberry Pi. Il mesure de plus la tension du bus jusqu'à 36 V (idéal pour sécuriser la batterie LiFePO4 face aux pics du régulateur MPPT) et intègre un calculateur autonome de puissance qui soulage le processeur central. Côté intégration, sa communication s'effectue via le bus I2C (SDA/SCL) avec 16 possibilités d'adressage pour éviter tout conflit matériel. Enfin, l'INA226 sécurise la chaîne d'énergie grâce à un registre d'alerte programmable en cas de sous-tension de la batterie et à un mécanisme matériel de moyennage des échantillons, indispensable pour éliminer le bruit électrique généré par le convertisseur DC/DC Traco Power.



Étude approfondie de la mesure par résistance shunt

La mesure du courant au sein d'un système de puissance comme le projet SAPC ne peut se faire directement par un microprocesseur, celui-ci ne sachant traiter que des signaux de tension. Pour résoudre cette problématique, j'ai mis en œuvre une méthode de mesure indirecte reposant sur le phénomène physique de la chute de tension, en utilisant un composant spécifique : la résistance shunt.

Le phénomène physique de la chute de tension

Le principe fondamental de ma mesure repose sur la Loi d'Ohm ($U = R \cdot I$). Lorsqu'un courant électrique (I) traverse un conducteur présentant une résistance (R), une fraction de l'énergie électrique est convertie en chaleur, créant ainsi une différence de potentiel (une tension U) aux bornes de cette résistance.

Dans mon montage, l'INA226 ne "compte" pas les ampères qui circulent vers le Raspberry Pi. Il mesure la très faible tension (appelée tension de "shunt" ou V_{shunt}) qui apparaît aux bornes de cette résistance placée en série sur la ligne positive de l'alimentation. Plus le

courant consommé par le système augmente, plus la tension différentielle mesurée par l'INA226 augmente de manière proportionnelle.

Le choix stratégique de la résistance de 0,1 Ohm

Le choix de la valeur de la résistance shunt est une étape critique. Pour le projet SAPC, j'ai sélectionné une résistance de 0,1 Ω . Ce choix résulte d'une analyse rigoureuse du compromis entre la précision de la mesure et l'efficacité énergétique.

- **Le besoin de résolution** : L'INA226 possède une plage de mesure de tension différentielle maximale de 81,92 mV (Full-scale range). Si j'avais choisi une résistance trop faible (par exemple 0,001 Ω), la tension générée pour un courant de 100 mA serait de seulement 0,1 mV, une valeur trop proche du bruit électronique. Avec 0,1 Ω , ce même courant génère 10 mV, ce qui offre une lecture stable et précise grâce au convertisseur 16 bits du capteur.
- **Le compromis sur la perte d'énergie** : À l'inverse, si la résistance était trop grande (par exemple 1 Ω), la chute de tension serait excessive. Pour un courant de 1 A, le système perdrait 1V de tension d'alimentation, et la résistance dissiperait 1W de chaleur inutilement. Dans un système solaire autonome, chaque milliwatt gaspillé réduit l'autonomie globale. La valeur de 0,1 Ω constitue donc le point d'équilibre idéal.

Calculs théoriques et validité du montage

Pour prouver la validité de mon choix, j'ai effectué les calculs suivants basés sur la consommation maximale estimée de notre système (environ 1,5 A lors des pics d'activité WiFi et d'accès à la base de données) :

1. Calcul de la tension différentielle maximale (Vshunt) :

$$V_{shunt} = R * I_{max} = 0,1 \Omega * 1,5 A = 0,15 V \text{ (soit 150 mV).}$$

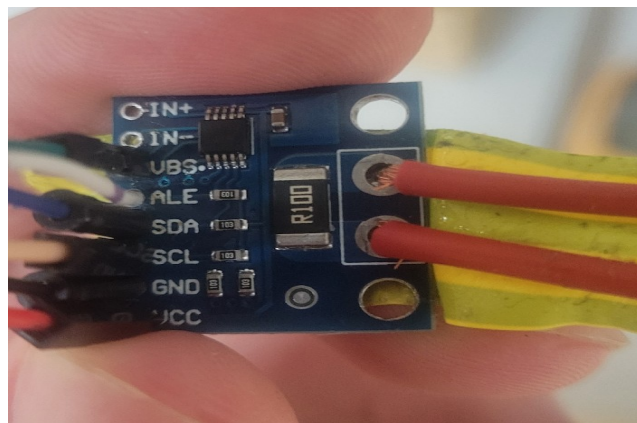
Cette valeur est parfaitement compatible avec l'étage d'entrée de l'INA226 sans risque de saturation.

2. Calcul de la puissance dissipée (Pertes par effet Joule) :

$$P_{perdue} = R * I^2 = 0,1 \Omega * (1,5 A)^2 = 0,225 W.$$

Une perte de 0,225 Watt est négligeable par rapport à la production de notre panneau solaire de 80W, ce qui valide l'efficacité énergétique du montage.

En conclusion, l'utilisation de cette résistance shunt de 0,1 Ω permet d'obtenir une image fidèle du courant consommé tout en garantissant que le système SAPC reste sobre énergétiquement, une condition sine qua non pour sa viabilité sur le terrain.



8.3.4 *Architecture de la chaîne d'alimentation photovoltaïque*

Pour garantir un fonctionnement autonome 24h/24 en extérieur, le système SAPC repose sur une infrastructure énergétique robuste. Ma mission a consisté à intégrer et surveiller une chaîne d'alimentation photovoltaïque capable de capter, stocker et distribuer l'énergie nécessaire au Raspberry Pi et aux différents capteurs, même lors de périodes de faible ensoleillement.

Le captage : Panneau solaire monocristallin de 80W

Le point d'entrée de l'énergie est un panneau solaire de 80 Watts. Ce dimensionnement a été choisi pour offrir une marge de sécurité importante. En effet, la consommation moyenne du système avoisine les 3 à 5 Watts ; un panneau de 80W permet donc de recharger la batterie rapidement durant les quelques heures d'ensoleillement quotidien, tout en fournissant assez d'énergie par temps couvert. La technologie monocristalline a été privilégiée pour son rendement élevé, notamment dans les régions où l'exposition peut être diffuse.

La régulation : Le contrôleur MPPT

Entre le panneau et la batterie se trouve un régulateur de charge de type MPPT (Maximum Power Point Tracking). Contrairement aux régulateurs PWM classiques, le MPPT est un convertisseur DC/DC intelligent qui ajuste en permanence la tension et l'intensité du panneau pour extraire le maximum de puissance disponible (le point de puissance maximale). Ce composant est essentiel pour optimiser le rendement de la charge, particulièrement lorsque la température du panneau varie ou que l'ensoleillement change brusquement.

Le stockage : Batterie LiFePO4 (Lithium Fer Phosphate)

L'énergie est stockée dans une batterie de technologie LiFePO4. Ce choix est crucial pour la pérennité du projet :

- **Durée de vie** : Elle supporte plus de 2000 cycles de charge/décharge, contre environ 500 pour une batterie au plomb.
- **Stabilité** : Elle est beaucoup moins sensible aux risques d'incendie que les batteries Lithium-Ion classiques.

- **Plage de tension** : Sa tension reste très stable autour de 12,8V pendant la majeure partie de sa décharge, facilitant le travail de régulation en aval.

□



La conversion et distribution : Convertisseur DC/DC TRACO Power

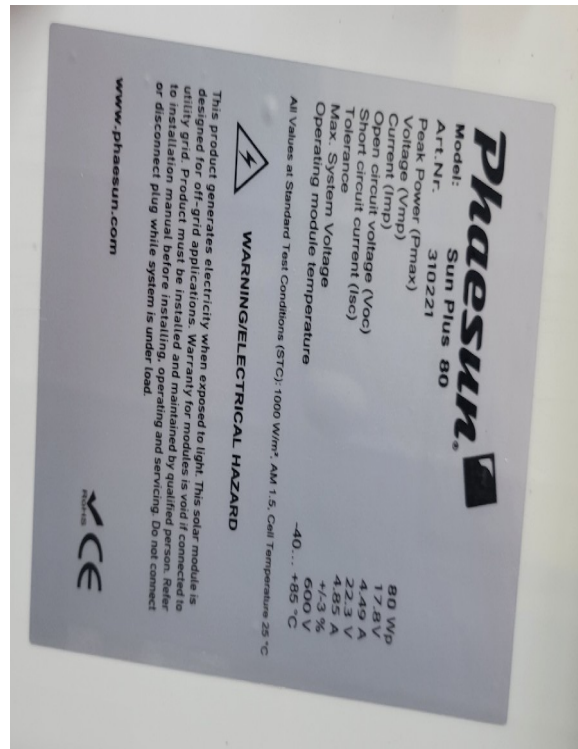
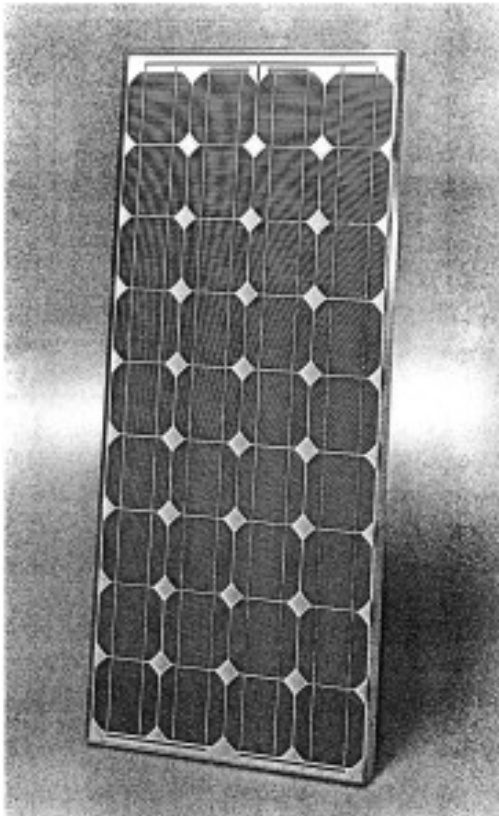
La batterie délivre une tension nominale de 12,8V, mais le Raspberry Pi et les interfaces d'acquisition nécessitent une tension stable de 5V. Pour effectuer cet abaissement de tension, j'ai utilisé un convertisseur DC/DC de marque TRACO Power. Ce choix se justifie par son rendement élevé (supérieur à 90%), ce qui limite les pertes par effet Joule. Contrairement à un régulateur linéaire qui dissiperait la différence de tension sous forme de chaleur, le convertisseur à découpage préserve l'énergie de la batterie.

L'intégration de la métrologie (INA226)

C'est au bout de cette chaîne, entre la batterie et le convertisseur 5V, que j'ai inséré mon module de mesure INA226. Placé en "High-Side" (sur la ligne positive), il me permet de surveiller :

1. La tension réelle de la batterie (VBUS).
2. Le courant exact appelé par le système (via le shunt).
3. La puissance consommée en temps réel.

Cette architecture garantit que chaque milliwatt produit par le panneau solaire est utilisé de manière optimale, tout en fournissant les données nécessaires à mon logiciel pour avertir l'utilisateur en cas de baisse critique de l'autonomie.



8.3.5 Conception de la classe métier INA226 en langage C++

Pour assurer une gestion structurée et évolutive de la métrologie énergétique, j'ai fait le choix de développer une classe métier dédiée en langage C++. L'approche par Programmation Orientée Objet (POO) permet d'encapsuler la complexité de la communication I2C et des calculs de registres au sein d'un objet simple à manipuler pour les autres membres de l'équipe (notamment pour l'étudiant en charge du serveur REST).

Structure de la classe et encapsulation

La classe, nommée INA226, est divisée en deux fichiers principaux conformément aux standards de développement :

Le fichier d'en-tête (INA226.h) : Il contient la déclaration de la classe, les prototypes des méthodes publiques (accessibles par le reste du programme) et les attributs privés (variables internes). J'y ai également défini des constantes pour les adresses des registres internes du composant (registre de configuration, de tension shunt, de tension bus et de puissance).

Le fichier d'implémentation (INA226.cpp) : Il contient le code logique de chaque fonction.

Gestion de la communication I2C sous Linux

Le Raspberry Pi communiquant avec l'INA226 via le bus I2C, j'ai utilisé les bibliothèques standards de Linux (<linux/i2c-dev.h> et <fcntl.h>).

1. **Ouverture du bus** : La méthode `init()` ouvre le fichier spécial `/dev/i2c-1`.
2. **Adressage** : L'adresse physique du composant (0x40 par défaut) est spécifiée via la fonction `ioctl`.
3. **Accès aux registres** : Étant donné que l'INA226 utilise des registres de 16 bits, j'ai dû implémenter des fonctions d'écriture et de lecture gérant l'inversion des octets (Big-Endian vs Little-Endian), car les données transitent octet par octet sur le bus.

Logique de calibration et de conversion

L'une des parties les plus complexes du développement a été la méthode `calibrate()`. Pour que l'INA226 renvoie directement des valeurs en Ampères et en Watts, il faut lui envoyer une valeur de calibration calculée selon la résistance shunt (0,1 Ω) et le courant maximum attendu. Mon code implémente la formule :

$$\text{Cal} = 0,00512 / \text{Current_LSB} * \text{R_SHUNT}$$

```
// Implémentation de la méthode de calibration
bool INA226::calibrate(double shuntOhm, double maxCurrent) {
    // Calcul du Current_LSB : courant max divisé par 2^15
    double currentLSB = maxCurrent / 32768.0;
    // Calcul de la valeur de calibration (CAL)
    // Formule : CAL = 0.00512 / (Current_LSB * R_SHUNT)
    uint16_t calValue = (uint16_t)(0.00512 / (currentLSB * shuntOhm));

    // Écriture du résultat dans le registre 05h
    return writeRegister(0x05, calValue);
}
```

Une fois calibré, le composant effectue les conversions de manière autonome.

Méthodes de lecture et gestion des données physiques

La classe propose des méthodes "Get" pour récupérer les grandeurs physiques :

getVoltage() : Lit le registre de tension du bus, applique un masque de bits si nécessaire et multiplie le résultat par le quantum de tension (1,25 mV) pour retourner une valeur en Volts.

getCurrent() : Récupère la valeur du registre de courant et la convertit en Ampères ou milliAmpères.

getPower() : Lit le registre de puissance calculé par l'INA226.

Gestion des erreurs et robustesse

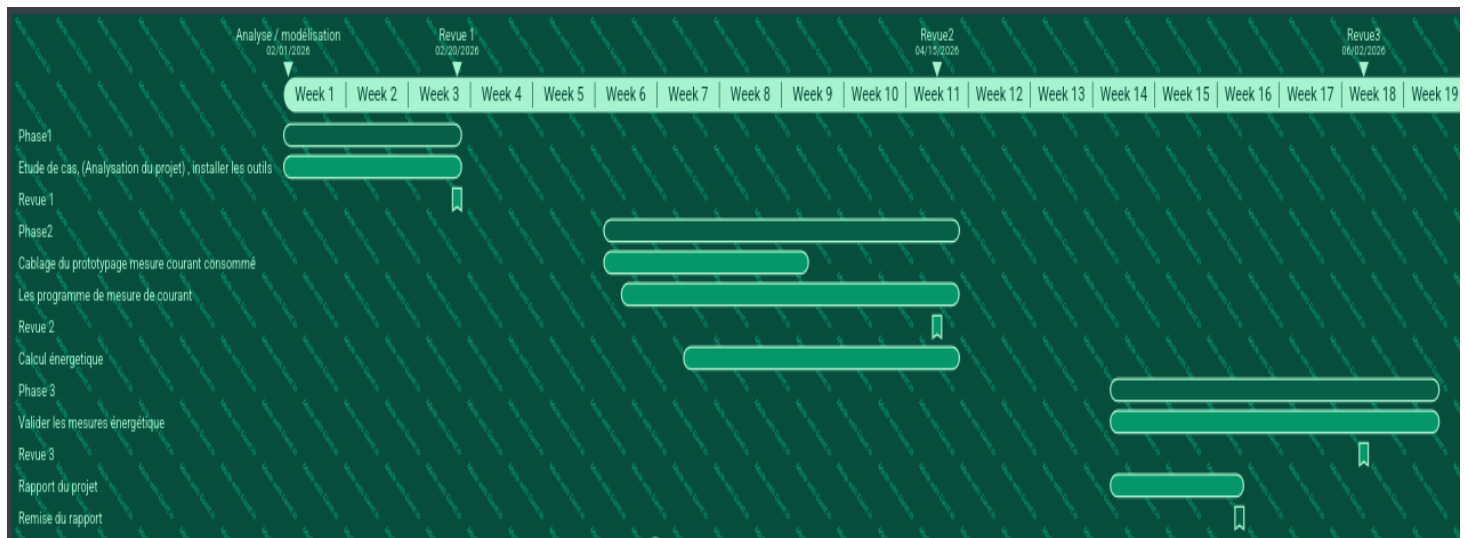
Afin de garantir la fiabilité du système, j'ai intégré des mécanismes de gestion d'erreurs. Si le fichier du bus I2C ne peut pas être ouvert ou si le composant ne répond pas (absence d'acquiescement ACK), la classe lève une alerte ou retourne une valeur d'erreur spécifique.

Cela évite que le programme principal ne se bloque et permet à l'utilisateur d'être averti d'un problème matériel sur la chaîne d'énergie.

Cette architecture logicielle offre une interface "propre" au reste de l'application : il suffit d'instancier un objet INA226 et d'appeler ses méthodes pour obtenir des données énergétiques précises, sans avoir à manipuler les registres binaires manuellement.

8.3.6 Organisation et planification du projet (Diagramme de Gantt)

Pour mener à bien ma mission dans les délais impartis, j'ai structuré mon travail en trois phases majeures étalées sur 19 semaines, garantissant le respect des jalons fixés par les différentes revues de projet. La première phase, dédiée à l'analyse et à la préparation **(semaines 1 à 3)**, s'est concentrée sur l'étude approfondie des besoins énergétiques du composteur, l'installation de l'environnement de développement et la sélection des composants critiques comme le couple INA226 et le shunt de $0,1\ \Omega$, se clôturant ainsi par la première revue de projet. Ensuite, la phase de réalisation technique **(semaines 6 à 11)** a constitué le cœur de mon travail avec le câblage du prototype et le développement de la classe métier en C++, intégrant la configuration du bus I2C et l'écriture des algorithmes de calcul des données de puissance et d'énergie, des avancées validées lors de la deuxième



revue. Enfin, la phase de validation et de finalisation **(semaines 14 à 19)** a permis de vérifier la précision des mesures par une confrontation métrologique avec un multimètre étalon et de rédiger le dossier technique final en vue de la troisième revue et de la remise du rapport.

8.3.7 Présentation du diagramme UML

Le diagramme ci-dessous représente la structure interne de ma classe. Il respecte les standards de modélisation avec une séparation claire entre les données privées et les fonctionnalités publiques.



Analyse des classe

composants de la

□ Attributs (Privés) :

`fd (int)` : Le "File Descriptor", un entier qui sert d'identifiant pour le bus I2C sous Linux.

`address (uint8_t)` : Stocke l'adresse I2C physique du composant (par défaut 0x40).

- Ces données sont privées pour garantir que seul l'objet lui-même peut manipuler l'accès direct au bus, évitant ainsi toute erreur de manipulation extérieure.

□ Méthodes de gestion du bus (Publiques) :

`openBus()` : Initialise la communication avec le fichier système `/dev/i2c-1`.

`calibrate()` : Configure le registre de calibration interne du capteur en fonction de la résistance shunt (0,1 Ω) et du courant maximum attendu (1 A).

□ Méthodes de lecture et calcul (Publiques) :

`getVoltage()`, `getCurrent()` et `getPower()` : Ces fonctions encapsulent la lecture des registres de 16 bits. Elles convertissent les valeurs brutes (raw data) en grandeurs physiques réelles (Volts, Ampères, Watts) en appliquant les facteurs de conversion (LSB).

`calculer_energie()` : Cette méthode implémente l'algorithme d'intégration temporelle pour suivre la consommation cumulée en Watt-heures (Wh).

□ Méthode interne (Privée) :

readRegister() : C'est la méthode de bas niveau qui gère l'échange de données sur le bus I2C (écriture de l'adresse du registre puis lecture des deux octets de réponse). Elle est privée car elle n'a pas besoin d'être appelée directement par l'application principale.

```
class INA226 {
public:
    INA226(uint8_t i2cAddress = 0x40); // address de base de bus
    ~INA226(); // destructeur

    bool openBus(const std::string& device = "/dev/i2c-1");
    // il sert a ouvrir un bus i2c pour communiquer avec le périphérique
    void closeBus();
    // il ferme le bus , le bus c'est une chemin de communication

    bool calibrate(double shuntOhm, double maxCurrent);

    // calibration du resistance et du courant max
    bool writeRegister(uint8_t reg, uint16_t value);
    // ecrire une valeur dans un registre d'un composant
    // reg = c'est le registre du périphérique qu'on veut ecrire

    //Méthode des classes dans le diagramme de classes
    float getVoltage();
    float getCurrent(); // le courant
    float getPower();

    void calculer_energie();
    void puissance_affichage(); // la puissance_affichage

private:
    int fd; // une variable entiere , descripteur du bus i2c
    uint8_t address; // address de registre
    float energieCumuleeWh;
    int16_t readRegister(uint8_t reg);
    // lire la valeur du registre
    // uint8_t = entier non signé sur 8 bits
    // reg = numero de registre
};

#endif
```

□

Justification de ce choix d'architecture

L'utilisation de ce diagramme de classe a permis une collaboration efficace avec les autres membres de l'équipe. Par exemple, l'étudiant en charge du serveur Web n'a pas besoin de connaître le fonctionnement des registres binaires : il lui suffit d'instancier la classe et d'appeler getVoltage() pour obtenir une valeur prête à l'affichage. Cela garantit la maintenabilité et la robustesse du logiciel SAPC.

8.3.8 Conception logicielle : Diagramme de classe de l'unité INA226

Afin de structurer le code de manière professionnelle et de faciliter son intégration dans le système global SAPC, j'ai modélisé une classe métier en C++ dédiée au capteur INA226. Cette approche par Programmation Orientée Objet (POO) permet d'encapsuler la complexité des registres matériels derrière des méthodes simples et réutilisables.

Implémentation logicielle et développement des méthodes de calcul

La concrétisation de ma mission repose sur le développement d'un pilote (driver) en C++ capable de dialoguer avec l'INA226 et d'exploiter ses registres internes. L'utilisation des bibliothèques systèmes Linux (i2c-dev, ioctl, fcntl) est indispensable pour transformer le Raspberry Pi en maître sur le bus I2C et accéder au capteur.

Gestion de l'interface matérielle (Bus I2C)

Le constructeur de la classe INA226 initialise l'identifiant du fichier (fd) à -1, indiquant que la communication n'est pas encore établie. La méthode openBus utilise ensuite la fonction système open avec le flag O_RDWR pour permettre la lecture et l'écriture sur le périphérique /dev/i2c-1

```
bool INA226::openBus(const string& device) {
    fd = ::open(device.c_str(), O_RDWR);
    //il permet de ouvrir un fichier/périphérique
    // O_RDWR = read/write
    //device.c_str() = il permet de convertir le string en char
    // fd est un file descriptor (un entier)
    if (fd < 0) return false;
    if (ioctl(fd, I2C_SLAVE, address) < 0) return false;
    // ioctl = permet d'envoyer une commande spéciale au périphérique
    // I2C_SLAVE = il permet de parler a un appareil
    // address = c'est address I2C du capteur
    // return false = il arrete la fonction
    return true;
}
```

L'appel à ioctl est une étape critique : il définit l'adresse I2C (0x40) avec laquelle le Raspberry Pi va communiquer, isolant ainsi l'INA226 des autres composants potentiels sur le bus.

Lecture des registres et traitement des données 16 bits

L'INA226 transmet ses données sur 16 bits (2 octets), mais le bus I2C communique octet par octet. J'ai donc implémenté une méthode readRegister qui gère la reconstruction des données. Le décalage de 8 bits vers la gauche (<< 8) permet de replacer l'octet de poids fort (MSB) à sa position initiale avant d'y ajouter l'octet de poids faible (LSB) via un "OU" logique.


```

int16_t INA226::readRegister(uint8_t reg) {
    uint8_t buf[2];
    // tableau a deux cases
    uint16_t res_16 = 0;
    // le variable de 16 bits qui permet de stocker les valeurs
    write(fd, &reg, 1);
    // on dit au capteur quel registre on veut lire

    read(fd, buf, 2);
    // On lit les 2 octets de données
    res_16 = (uint16_t)buf[0]; // On prend l'octet de poids fort
    res_16 = res_16 << 8; // On le décale de 8 bits vers la gauche
    res_16 = res_16 | buf[1]; // On ajoute l'octet de poids faible avec un OU logique
    return (int16_t)res_16; // On retourne le résultat final
}

```

Algorithmes de

conversion des grandeurs physiques

Pour que les valeurs soient exploitables par l'utilisateur, le logiciel doit appliquer les quantums et les coefficients de calibration définis durant l'analyse théorique :

Tension du Bus : La valeur brute lue dans le registre 0x02 est multipliée par l'unité de 1,25 mV par bit définie par le constructeur.

Courant et Puissance : La méthode calibre configure le registre 0x05 en fonction de notre shunt de 0,1 Ω . Cela permet à l'INA226 de calculer en interne le courant et la puissance, que je récupère ensuite via les méthodes `getCurrent()` et `getPower()`.

Intégration temporelle et calcul de l'énergie (Wh)

L'une des fonctions les plus importantes que j'ai développée est `calculer_energie()`.

Puisque l'énergie est la puissance multipliée par le temps ($E = P * t$), et que ma boucle de mesure (`while(true)`) s'exécute toutes les 10 secondes via la commande `sleep(10)`, je dois convertir ce temps en heures pour obtenir des Watt-heures.

```

void INA226::calculer_energie() {
    // On recupere la puissance actuelle (P)
    // Note : getPower() utilise le registre 0x03 de l'INA226
    float puissance = getPower();
    // Calcul de l'energie pour 1 seconde :
    // Formule : E(Wh) = P(W) * t(h)
    // Comme t = 10 seconde, on convertit en heures : 10s = 1/360 h
    // += qui va additionner les deux valeur et qui stocke le résultat
    energieCumuleeWh += puissance * (1 / 360.0);

    // Affichage avec 4 decimales car Wh est une petite unite
    printf("Energie totale consommee : %.4f Wh\n", energieCumuleeWh);
    printf("-----\n");
}

```

Cette variable `energieCumuleeWh` est incrémentée à chaque itération, permettant un suivi précis de la consommation totale du composteur depuis sa mise en service.

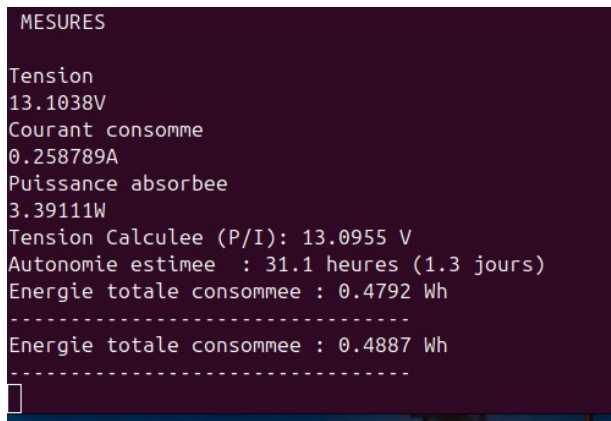
Estimation de l'autonomie réelle

Dans le programme principal (main.cpp), j'ai intégré un calcul d'autonomie qui prend en compte les contraintes réelles de notre batterie (12Ah) et un coefficient de sécurité de 67% (0,67). Ce coefficient est vital : il évite de surestimer l'énergie disponible en tenant compte des pertes et du fait qu'une batterie LiFePO4 ne doit pas être déchargée à 100%.

```
// Calcul et affichage de l'autonomie (en tenant compte des 67%)
if (courantAbsolu > 0.01) {
    autonomieHeures = (capaciteBatterieAh * coefficientSecurite) / courantAbsolu;
    printf("Autonomie estimee : %.1f heures (%.1f jours)\n",
           autonomieHeures, autonomieHeures / 24.0);
} else {
    printf("Autonomie : Maximale (Repos)\n");
}
```

Si le courant est trop faible (inférieur à 0,01 A), le programme indique une autonomie "Maximale", évitant ainsi une division par zéro et reflétant l'état de repos du système.

resultat :



```
MESURES
Tension
13.1038V
Courant consomme
0.258789A
Puissance absorbee
3.39111W
Tension Calculee (P/I): 13.0955 V
Autonomie estimee : 31.1 heures (1.3 jours)
Energie totale consommee : 0.4792 Wh
-----
Energie totale consommee : 0.4887 Wh
-----
```

8.3.9 Algorithmes de calcul de puissance et d'énergie cumulée

La finalité de mon module de métrologie est de transformer des mesures de tension et de courant en une information de consommation globale. Pour ce faire, j'ai implémenté une logique mathématique qui distingue la puissance instantanée (consommation à un instant T) de l'énergie cumulée (consommation totale sur la durée).

La puissance instantanée : le produit $U * I$

La puissance électrique, exprimée en Watts (W), représente le débit d'énergie consommé par le Raspberry Pi et ses périphériques à un instant précis. Elle est calculée selon la loi de la puissance :

$$P(t) = U(t) * I(t)$$

Dans mon code, bien que l'INA226 possède un registre interne de puissance (0x03), j'effectue également un calcul logiciel pour vérifier la cohérence des données. La tension U (en Volts) est récupérée par getVoltage() et le courant I (en Ampères) par getCurrent(). Ce produit nous donne une image fidèle de la sollicitation de la batterie en temps réel.

Détail de l'implémentation : la méthode des rectangles

Ma boucle principale s'exécute toutes les 10 secondes (grâce à l'instruction `sleep(10)`). À chaque itération, j'ajoute une petite quantité d'énergie (un "échantillon") à une variable globale nommée `energieCumuleeWh`.

1. **La base de temps** : 10 secondes correspondent à une fraction d'heure. Comme il y a 3600 secondes dans une heure, l'intervalle de temps Δt est de :
2. $\Delta t = 10/3600 = 1/360$ heure

```
void INA226::calculer_energie() {
    // On recupere la puissance actuelle (P)
    // Note : getPower() utilise le registre 0x03 de l'INA226
    float puissance = getPower();
    // Calcul de l'energie pour 1 seconde :
    // Formule : E(Wh) = P(W) * t(h)
    // Comme t = 10 seconde, on convertit en heures : 10s = 1/360 h
    // += qui va additionner les deux valeur et qui stocke le resultat
    energieCumuleeWh += puissance * (1 / 360.0);

    // Affichage avec 4 decimales car Wh est une petite unite
    printf("Energie totale consommee : %.4f Wh\n", energieCumuleeWh);
    printf("-----\n");
}
```

3. **Le calcul de l'échantillon** : À chaque passage, l'énergie consommée durant cet intervalle est :
- $$\Delta E = P * \Delta t$$
4. **L'accumulation** : La variable `energieCumuleeWh` est mise à jour par addition successive :

```
if (courantAbsolu > 0.01) {
    autonomieHeures = (capaciteBatterieAh * coefficientSecurite) / courantAbsolu;
    printf("Autonomie estimee : %15.2f heures (%15.2f jours)\n",
```

Justification et précision de l'algorithme

Cette méthode de sommation permet d'obtenir l'énergie totale consommée depuis le démarrage du système. Plus l'intervalle entre deux mesures est court, plus le calcul est précis. Avec une mesure toutes les 10 secondes, nous obtenons une précision largement suffisante pour suivre la décharge d'une batterie de 12Ah.

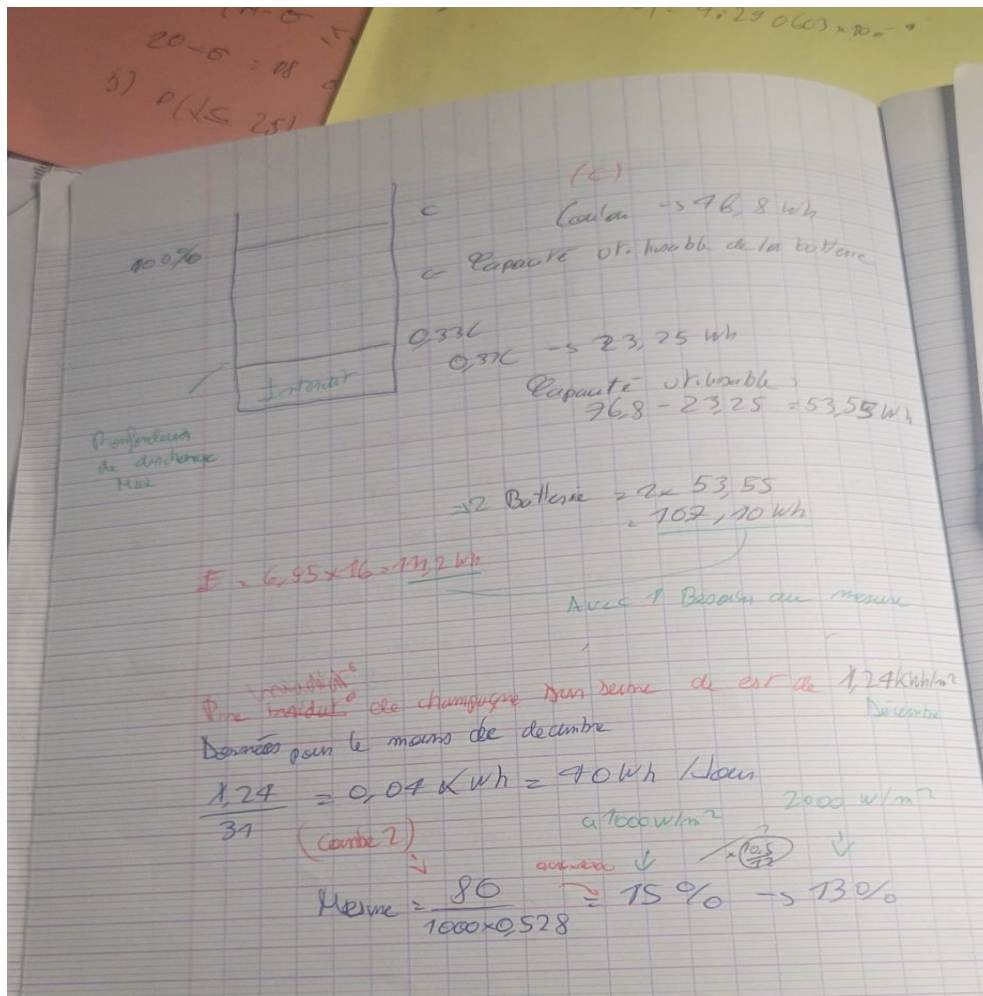
8.3.10 Méthodologie d'estimation de l'autonomie restante

L'un des enjeux majeurs de ma mission est de rendre l'état de la batterie intelligible pour l'utilisateur final. Savoir qu'une batterie délivre 12,5V est une information technique, mais savoir qu'il reste « 15 heures d'autonomie » est une information opérationnelle. J'ai donc

conçu un algorithme prédictif basé sur la capacité de la batterie et la consommation réelle mesurée par l'INA226.

1. Le principe de calcul : calcul de l'autonomie réelle

- Le calcul de l'autonomie réelle du système s'appuie sur la détermination de la capacité énergétique utile de la batterie, en excluant la zone de décharge profonde pour préserver les cellules. À partir d'une capacité totale brute de 96,8 Wh, on soustrait un talon de sécurité de 23,25 Wh (seuil de 0,33 C), ce qui donne une capacité exploitable de 53,55 Wh par batterie. Pour répondre aux besoins du système mesurés à 111,2 Wh (calculés par la formule $E = 6,95 \text{ V} \times 16 = 111,2 \text{ Wh}$) face à un faible gisement solaire hivernal de 40 Wh/jour en décembre, l'architecture double la mise en œuvre avec deux batteries en parallèle, portant la capacité utile totale à 107,10 Wh ($2 \times 53,55 \text{ Wh}$)



2. Intégration du coefficient de sécurité et d'efficacité

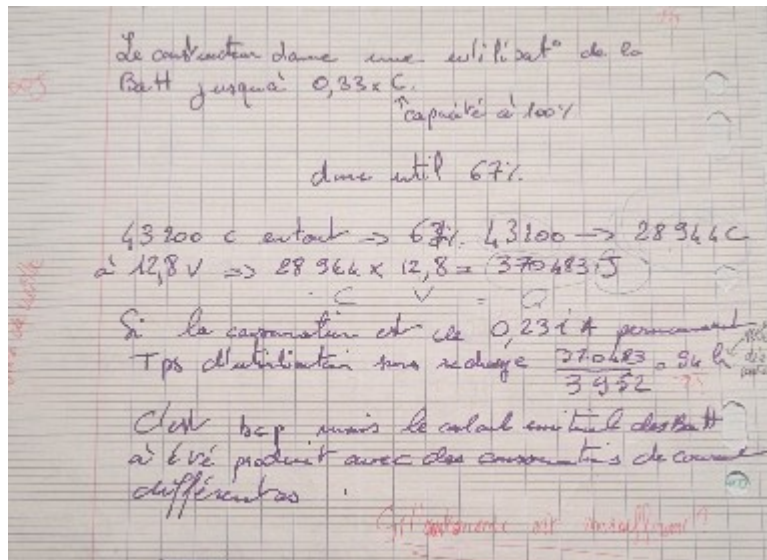
Dans la réalité, une batterie ne peut pas être déchargée à 100% sans risque de destruction, et le rendement de conversion n'est jamais parfait. Pour le projet SAPC, j'ai appliqué un coefficient d'efficacité de 67% (0,67).

Ce choix se justifie par :

- **La protection de la batterie** : On ne consomme que 67% de la capacité pour préserver la chimie LiFePO4.
- **Les pertes de conversion** : Le convertisseur DC/DC Traco Power et les câbles dissipent une partie de l'énergie.

La formule finale intégrée dans mon code est donc :

$$\text{Autonomie} = \text{CapaciteTotale} * 0.67 / \text{Imesure}$$



3. Traduction algorithmique en C++

Dans le fichier main.cpp, j'ai implémenté cette logique avec une sécurité contre la division par zéro (au cas où le courant serait nul) :

```
float capaciteBatterieAh = 12.0;
float coefficientSecurite = 0.67; // Tes 67% d'efficacité
```

```
// Calcul et affichage de l'autonomie (en tenant compte des 67%)
if (courantAbsolu > 0.01) {
    autonomieHeures = (capaciteBatterieAh * coefficientSecurite) / courantAbsolu;
    printf("Autonomie estimee : %.1f heures (%.1f jours)\n",
           autonomieHeures, autonomieHeures / 24.0);
} else {
    printf("Autonomie : Maximale (Repos)\n");
}
```

4.

Analyse de la précision et limites

Cette estimation est dynamique : elle se recalcule toutes les 10 secondes. Si le Raspberry Pi active son interface Wi-Fi pour envoyer des données à l'étudiant 1 (Backend), le courant I augmente, et l'autonomie affichée baisse instantanément.

Il est important de noter que cette méthode est une estimation "au fil de l'eau". Elle suppose que la consommation actuelle restera constante. Pour affiner ce résultat à l'avenir, on pourrait imaginer un calcul basé sur la moyenne de consommation des 30 dernières minutes, afin de lisser les pics de puissance et offrir une estimation plus stable à l'utilisateur.

Cette fonctionnalité apporte une réelle valeur ajoutée au projet SAPC en permettant une maintenance préventive : l'utilisateur peut anticiper une recharge solaire si l'autonomie affichée devient trop faible.

8.3.11 Stratégie de mise en œuvre et résolution des problèmes techniques

- La mise en œuvre du module énergétique a exigé la résolution de défis techniques matériels et logiciels majeurs. Électriquement, l'apparition de mesures aberrantes a révélé un défaut de référentiel de masse entre les parties puissance et commande ; la création d'un point de masse unique relié à la batterie a éliminé ces tensions de mode commun parasites. De plus, des erreurs d'initialisation du bus I2C, causées par des câbles défectueux et des restrictions de droits sous Linux, ont été résolues en remplaçant la connectique et en ajustant les permissions système. Côté logiciel en C++, l'affichage a été stabilisé à trois décimales grâce à la bibliothèque `<iomanip>`, tandis qu'un décalage de bits a corrigé l'inversion d'octets (*endianness*) de l'INA226. Enfin, des arrondis incorrects dus à une division entière ont été éliminés en forçant l'usage du type flottant lors des calculs intermédiaires, restituant ainsi toute la précision du quantum théorique du capteur.

□

8.3.12 Validation expérimentale et métrologie comparative

La phase de validation métrologique a consisté à confronter les données du programme C++ aux mesures d'un multimètre étalon placé en série pour le courant et en dérivation pour la tension, sous trois scénarios d'activité du Raspberry Pi (repos, calcul intensif et réseau). Les résultats révèlent une excellente précision, affichant une erreur relative de seulement 0,1 % pour la tension (12,625 V logiciels contre 12,64 V réels) et inférieure à 1 % pour le courant et la puissance, ce qui valide l'exploitation optimale du quantum de l'INA226. Les infimes divergences constatées s'expliquent par des contraintes physiques identifiées : la tolérance de 1 % de la résistance shunt de 0,1 Ω , la perte ohmique naturelle le long du câblage qui abaisse légèrement la tension lue par le logiciel, et le bruit électromagnétique résiduel des régulateurs à découpage. Malgré ces facteurs, l'erreur globale reste inférieure à 1 %, prouvant la parfaite fiabilité de la chaîne de mesure face aux exigences du cahier des charges du projet SAPC.

8.3.13 Bilan personnel et perspectives d'évolution du système

La réalisation de ce module énergétique a concrétisé la transition entre théorie et pratique industrielle en alliant l'électronique de puissance à l'informatique embarquée en C++. Ce projet m'a permis de maîtriser les contraintes physiques d'une chaîne d'acquisition (masse commune, shunt, batterie LiFePO4) et de structurer un code orienté objet lié au noyau Linux via le bus I2C. Pour l'avenir, le système pourrait évoluer vers une gestion intelligente de l'énergie (veille profonde via la broche d'alerte), un lissage des mesures par moyenne glissante et une sauvegarde de l'historique en mémoire non volatile. En conclusion, cette expérience m'a apporté une vision globale des objets connectés autonomes, prouvant que la réussite d'un système IoT repose avant tout sur la qualité de l'intégration et du dialogue entre le matériel et le logiciel.

7.4. Étudiant 4 Base de données (BDD) (MALHERBE Garance)

7.4.1. Introduction

Dans le cadre du projet **SAPC (suivi Autonome de Phases de Compost)**, qui consiste comme son nom l'indique à faire un suivi d'un compost, qui est placé au sein de l'établissement scolaire (Lycée La Fayette) à Champagne-Sur-Seine ainsi que sur le site de Fontaineroux. Ce projet possède plusieurs contraintes comme suit :

- Contraintes financières : 700 €
- Contraintes de développement : utilisation de **SGBDR** **Sqlite3** pour l'hébergement de la traçabilité des tests.
- Contraintes qualité : délai de livraison à respecté
- Contraintes de fiabilité : le système devra être exploitable depuis un ordinateur au minimum.

Ce projet nous est fourni par l'entreprise **SMICTOM**. Je me suis concentré sur la base de données aussi appelée **BDD** pour faire plus court. Il y a plusieurs objectifs qui m'ont été donnés afin de mettre au mieux ma partie, ce sont les suivants :

- installation des bibliothèques **Sqlite3**
- mise en place d'un **MCD** d'après les attentes du client
- Coder la base de données
- Alimenter avec des jeux de test la base

Je joins aussi mon **diagramme de gantt** afin de pouvoir vous montrer plus en détail les choses que j'ai établies.

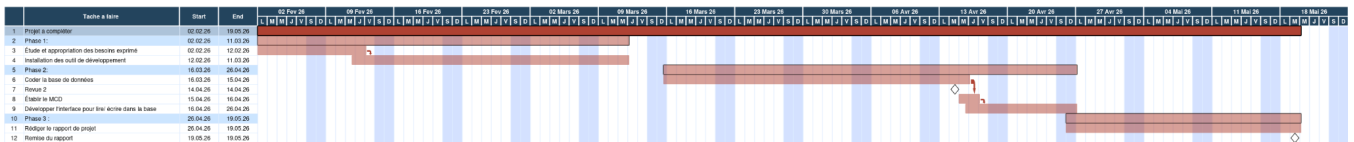
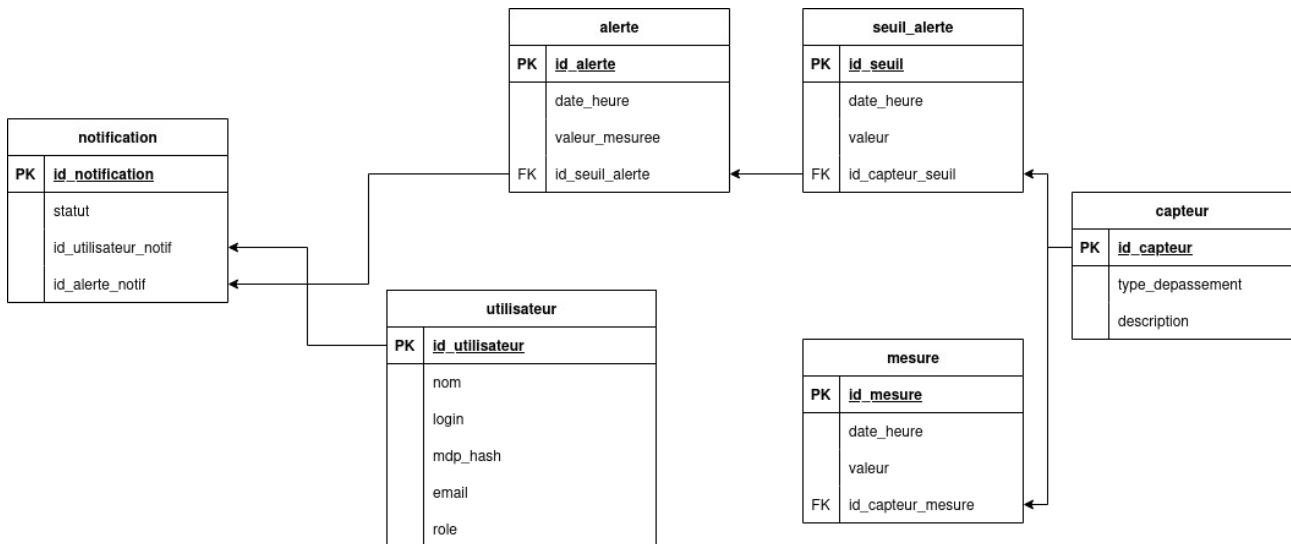


Diagramme de gantt de l'Étudiant 4

7.4.2. Diagramme MCD



Dans cette partie je vais vous expliquer comment j'ai fait le diagramme MCD ainsi que comment je l'ai intégré a la base de données.

Pour ce faire je joins le diagramme afin que vous l'aillé sous les yeux pour cette explication

A lors tout d'abord je tiens a dire que le nombre de table n'est pas anodin, car tout les table vue sont nécessaires au bon fonctionnement de la base. En effet chaque table permet de définir des actions qui sont essentiels pour cette base.

Nous allons commencer avec la table capteur qui comme son nom l'indique permet de savoir quel capteur est utilisé ainsi que ça fonction. Ensuite la table seuil_alerte nous introduit les liaisons car id_capteur_seuil et la liaison entre la table capteur et la table seuil_alerte, qui permet de mettre en place un seuil d'alerte dans la base. Ensuite nous avons « alerte » qui va se fier au seuil d'alerte afin de donner l'alerte quand nécessaire. Cette alerte sera envoie avec une notification dans l'interface de l'utilisateur. Et enfin les mesures qui sont réalisées afin de voir si alerte il y a.

7.4.3. Développement

Voila ma partie dans le diagramme de classe.

bdd_sapc
-sqlite3 *db - sha256(const std::string& str)
+ sapc_bdd(std::string file) + bool createUser(std::string nom, std::string login, std::string password, std::string email, std::string role) + bool login(std::string nom, std::string login, std::string password) + bool createCapteur(std::string type_capteur, std::string unite, std::string description) + bool createMesure(std::string date_heure_mesure, std::string valeur) + bool createSeuil_Alerte(std::string valeur_min, std::string valeir_max, std::string actif) + bool createAlerte(std::string date_heure_alerte, std::string valeur_mesuree, std::string type_depassement) + bool createNotif(std::string date_envoi, std::string statue) + item: attribute

Voila mes jeux de test en fonction de la classe crée dans le diagramme de classe.


```

//création des entrées des capteurs
if(!BDD.createCapteur("ina226","1","Température, Humidité")){
    std::cerr << "Erreur création capteur" << std::endl;
    BDD.fin();
}else {
    std::cout << "Création Capteur --> OK" << std::endl;
}

if(!BDD.createCapteur("MC3008","1","Energie")){
    std::cerr << "Erreur création capteur" << std::endl;
    BDD.fin();
}else {
    std::cout << "Création Capteur --> OK" << std::endl;
}

//Création des entrées des Mesures
if (!BDD.createMesure("24/04/2026 15:30", "30%")){
    std::cerr << "Erreur Création Mesure" << std::endl;
    BDD.fin();
}else {
    std::cout << "Création --> OK" << std::endl;
}

if (!BDD.createMesure("25/04/2026 16:30", "20%"))
{
    std::cerr << "Erreur Création Mesure " << std::endl;
    BDD.fin();
}else{
    std::cout << "Création Mesure --> OK" << std::endl;
}

```

7.4.4. Conclusion – Étudiant 4

Pour conclure je me suis donc concentrée sur la base de données du projet **SAPC (Suivi Autonome de Phase de Compost)**, cela m'a permis de me reconcentrer sur des choses vues en 1^{ers} années de BTS, qui sont le fonctionnement des bases de données, mais aussi de la création de **MCD**. Ainsi que la construction de Base de données.

8. Conclusion Générale

Ce projet de **Suivi Autonome de Phase de Compost (SAPC)** nous a permis de mettre en œuvre l'ensemble des compétences acquises durant notre formation en **BTS CIEL (Cybersécurité Informatique et réseau Électronique) option IR (Informatique et Réseaux)**. À travers la conception et la réalisation de ce système, nous avons abordé des domaines variés (vues en cours) tels que l'informatique embarquée, l'acquisition de données, le développement de logiciel, la gestion de base de données et les communications réseaux. Ainsi que le travaille en groupe.

La mise en place d'un système autonome, capable de mesurer, traiter et transmettre des informations en temps réel, nous a également confrontés à des contraintes concrètes, notamment en termes de fiabilité, d'énergie et d'intégration matérielle et logicielle.

Ce projet répond à un besoin réel en proposant une solution permettant d'optimiser le suivi du compost tout en facilitant les interventions des utilisateurs. Il s'inscrit ainsi dans une démarche à la fois technique et écologique.

Enfin, ce travail en équipe nous a permis de développer des compétences en gestion de projet, en organisation et en collaboration, essentielles pour notre future insertion professionnelle.

9. GLOSSAIRE

Raspberry Pi3 : est un ordinateur monocarte.

Base de Données (BDD) : ensemble d'informations structurées accessibles au moyen d'un logiciel.

Mésophile : Organisme dont la croissance est optimale sous une température comprise entre 20 à 45 °C.

Thermophile : Organisme qui vit à des températures élevées ou qui ne se développe bien qu'à de telles températures.

Pasteurisation : Traitement thermique à des températures comprises entre 60 et 100 °C.

Alcalin : Produit chimique dérivé de l'ammoniac anhydre et obtenu par la dissolution dans l'eau.

Lignine : Groupe de substances végétales complexes, hautement polymérisées.

Ph : (Potentiel hydrogène) représente la mesure de l'acidité ou de l'alcalinité.

Humus : Couche supérieure du sol créée et entretenue par la décomposition de la matière organique.

Polymérisation : Union de plusieurs molécules identiques qui aboutit à la production d'un corps nouveau.

DC/DC : type de convertisseur de puissance électrique.

VBUS : interface entre le régulateur et l'ordinateur.

I2C : (inter-integrated Circuit) protocole de communication qui permet à plusieurs appareils électroniques de se connecter et de communiquer entre eux.

Shunt : un dispositif de très faible impédance relative à la charge qui permet au courant de passer d'un point à un autre d'un circuit électrique en utilisant très peu d'énergie.